DOSSIER DE PROJET

BiblioTech

Système d'Information de Gestion de Bibliothèque (SIGB)

Auteur : Grégory Sergent

Formation : CDA – Concepteur Développeur d'Applications

Organisme : IPSSI Grande École d'Informatique

Période : Janvier → Juillet 2026

Stack technique : Symfony 7.3 + PHP 8.3 + React 19 + Vite + TypeScript + PostgreSQL 16

Déploiement : Render (backend + frontend) + Supabase (base de données)

CI/CD : GitHub Actions — 3 jobs (PHPUnit + TypeScript + Docker Build)

Table des matières

Cahier des Charges Fonctionnel :

Application "BiblioTech Desktop"

Sommaire

I. Contexte Métier

I.1 Contexte et Problématique

I.2 Commanditaire

II. Objectifs du Projet

III. Périmètre Fonctionnel

III.1 Exigences Fonctionnelles

III.2 Exigences Non Fonctionnelles

IV. Exigences Techniques

V. Contraintes et Enjeux du Projet

1. Calendrier et Jalons

2. Risques Identifiés

3. Critères de Succès

VI. Glossaire

Jalon 2 – Méthodologie & Conception UI/UX

Partie 1 – Méthodologie et organisation du projet

1.1 Méthode de gestion

1.2 Macro-planning (jusqu'à juin 2026)

1.3 Suivi des tâches

1.4 Gestion du code source (Git)

1.5 CI/CD planifié

Partie 2 – Conception UI/UX

2.1 Sitemap (écrans principaux)

2.2 Zoning (écran type)

2.3 Wireframes (basse fidélité)

2.4 Charte graphique

2.5 Maquettes graphiques (haute fidélité)

2.6 Considérations UX

Jalon 3 – Modélisation de la Base de Données

0) Évolution du périmètre fonctionnel

1) Introduction – Démarche MERISE

2) Dictionnaire des données

2.1 Entité : bibliotheques

2.2 Entité : utilisateurs

2.3 Entité : livres

2.4 Entité : exemplaires
2.5 Entité : prets
2.6 Entité : demandes_migration
3) MCD – Modèle Conceptuel de Données
Relations et cardinalités
4) MLD – Modèle Logique de Données
5) MPD – Modèle Physique de Données (PostgreSQL)
Script SQL – CREATE TABLE
6) Justifications de modélisation

Jalon 4 – Conception de l'Application & Architecture

1) Introduction
2) Diagrammes de cas d'utilisation (Use Cases UML)
2.1 Acteurs identifiés
2.2 Diagramme
2.3 Cas d'utilisation principaux
3) Diagrammes de séquence
3.1 Séquence 1 – Enregistrer un prêt
3.2 Séquence 2 – Ajouter un livre via ISBN (Google Books API)
3.3 Séquence 3 – Connexion et authentification JWT
4) Diagramme de classes
4.1 Vue d'ensemble
4.2 Correspondance Entities ↔ MPD (Jalon 3)
4.3 Principes appliqués
5) Architecture multi-couches
5.1 Pattern MVC avec Symfony
5.2 Architecture n-tiers
5.3 Conteneurisation Docker
5.4 API externe – Google Books
5.5 Sécurité – Vue d'ensemble
6) Endpoints de l'API REST
6.1 Authentification
6.2 Bibliothèques
6.3 Utilisateurs
6.4 Livres
6.5 Exemplaires
6.6 Prêts
6.7 Demandes de migration
6.8 Statistiques
7) Stratégie de tests
7.1 Objectifs
7.2 Types de tests et outils
7.3 Couverture cible par composant
7.4 Organisation des tests
7.5 Base de données de test

7.6 Intégration continue (CI)
8) Gestion des erreurs API
8.1 Format standard des réponses d'erreur
8.2 Codes HTTP utilisés
8.3 Catalogue des erreurs métier
9) État d'avancement du développement

Jalon 5 – Développement, Sécurité & Tests

1) Introduction
2) Ce qui a été développé
2.1 Vue d'ensemble
2.2 Entités Doctrine développées
2.3 Endpoints implémentés
2.4 Services métier
2.5 Frontend React (état d'avancement)
3) Fonctionnalités avancées implémentées
3.1 Système de premier login (mot de passe + CGU)
3.2 Protection anti-brute force
3.3 Versionnement des CGU & CMS intégré
3.4 Gestion des prêts perdus
3.5 Durée de prêt configurable par bibliothèque
4) Sécurité
4.1 Authentification JWT & Refresh Token
4.2 Contrôle d'accès (RBAC)
4.3 Protection OWASP Top 10
4.4 Validation des entrées
4.5 Conformité RGPD
5) Tests
5.1 Stratégie appliquée
5.2 Tests unitaires — `PretService`
5.3 Tests unitaires — `GoogleBooksService`
5.4 Tests d'intégration — `AuthController`
5.5 Tests d'intégration — `LivreController`
5.6 Tests d'intégration — `PretController`
5.7 Résultats d'exécution
6) Infrastructure, CI/CD & Déploiement
6.1 Docker Compose
6.2 Migrations Doctrine
6.3 Variables d'environnement
6.4 CI/CD — état des lieux
7) État d'avancement global du projet
8) Conclusion
Annexes
Annexe A — Diagramme de classes
Annexe B — Diagramme des cas d'utilisation

Annexe C — Diagrammes de séquence

Jalon 6 – Déploiement, Sécurité & Mise en Production

1) Introduction
- Rappel des jalons précédents
- Ce que couvre le Jalon 6

2) Audit de sécurité — synthèse
- 2.1 Périmètre audité
- 2.2 Vulnérabilités corrigées (14 au total)
- 2.3 Risques résiduels documentés (acceptés)

3) Correctifs appliqués (Jalon 6)
- 3.1 Backend
- 3.2 Frontend

4) Infrastructure & Docker
- 4.1 Architecture des conteneurs
- 4.2 Dockerfile backend (`dev/backend/Dockerfile`)
- 4.3 Dockerfile frontend (`dev/frontend/Dockerfile`)
- 4.4 Docker Compose final

5) CI/CD — Pipeline GitHub Actions
- 5.1 Structure de la pipeline
- 5.2 Job backend — PHPUnit
- 5.3 Job frontend — TypeScript check
- 5.4 Job Docker Build & Push

6) Instructions de déploiement
- 6.1 Prérequis
- 6.2 Déploiement local (développement)
- 6.3 Déploiement production
- 6.4 Variables d'environnement de production

7) Stratégie de mise en production
- Approche retenue : Maintenance window + rolling Docker
- Stratégie blue/green (idéale pour production réelle)
- Rollback

8) Conformité RGPD
- Registre des traitements (Article 30)
- Droits des utilisateurs implémentés
- Mesures techniques

9) Bilan des tests
- Couverture PHPUnit
- Exécution locale
- Résultats CI
- Captures — CI et déploiement en production
- Tests de sécurité

10) Bilan global & perspectives
- Ce qui a été réalisé (6 jalons)
- Difficultés techniques surmontées
- Perspectives d'évolution

Cahier des Charges Fonctionnel :

Application "BiblioTech Desktop"

Auteur : Grégory Sergent
Organisme de formation : IPSSI Grande École d'Informatique
Objet : Jalon 1 – Numérisation et gestion des bibliothèques locales
Date : 30 avril 2026
Statut : Version 2.0

Sommaire

Cahier des Charges Fonctionnel : Application "BiblioTech Desktop" 1
I. Contexte Métier 2
  I.1 Contexte et Problématique 2
  I.2 Commanditaire 2
II. Objectifs du Projet 2
III. Périmètre Fonctionnel 2
  III.1 Exigences Fonctionnelles 2
  III.2 Exigences Non Fonctionnelles 3
IV. Exigences Techniques 3
V. Contraintes et Enjeux du Projet 3
  1. Calendrier et Jalons 3
  2. Risques Identifiés 3
  3. Critères de Succès 3
VI. Glossaire 4

I. Contexte Métier

I.1 Contexte et Problématique

Le secteur des bibliothèques municipales et associatives en France fait face à un défi de modernisation. Si les grandes structures disposent de progiciels complexes (SIGB), de

nombreuses petites et moyennes bibliothèques utilisent encore des méthodes manuelles ou des outils obsolètes pour le suivi de leur catalogue et de leurs usagers.
BiblioTech est une application desktop conçue pour simplifier la transition numérique de ces établissements. Le problème à résoudre est la perte d'informations liée au suivi manuel et le manque de visibilité sur l'état du stock en temps réel. L'application s'adresse exclusivement au **personnel de la bibliothèque** (bibliothécaires et bénévoles) pour centraliser la gestion des prêts, des retours et des relances.

I.2 Commanditaire

> ⚠ *Le commanditaire, la structure et les personnes mentionnées dans cette section sont entièrement fictifs. Ils ont été créés dans le cadre d'un exercice pédagogique.*

Nom	L'IPSSIthèque *(fictif)*
Type de structure	Bibliothèque associative — Aubusson, Creuse (23), commune rurale de ~3 200 habitants parmis les moins connectés de France *(fictif)*
Contact projet	Mme Isabelle Perrin, directrice de la bibliothèque *(fictif)*
Situation actuelle	Gestion des prêts sur registres papier et tableur Excel non partagé
Besoin exprimé	Disposer d'un outil numérique simple, installable sur les postes du personnel, pour centraliser catalogue, adhérents et prêts

> **Distinction commanditaire / utilisateurs finaux :**
> Le **commanditaire** est l'**association gestionnaire de L'IPSSIthèque** (qui finance et valide le projet). Les **utilisateurs finaux** sont les membres du personnel de la bibliothèque (bibliothécaires et bénévoles). Les usagers de la bibliothèque (lecteurs) ne sont pas utilisateurs de l'application : ils n'ont aucun accès direct à BiblioTech.

II. Objectifs du Projet

L'objectif est de livrer un outil robuste et sécurisé d'ici juin 2026. Les objectifs SMART sont :
● **Digitaliser 100% des flux d'emprunts** : Supprimer les registres papier au profit d'une base de données centralisée.
● **Automatiser l'inventaire** : Réduire de 80% le temps de saisie d'un nouveau livre grâce à l'interrogation d'API externes via l'ISBN.

● **Optimiser le suivi des retards** : Identifier instantanément les livres non rendus pour faciliter les relances.

● **Assurer la conformité RGPD avant la mise en production** : mettre en place un registre de traitement documenté, implémenter le droit à l'oubli dans l'interface d'administration, et hacher les mots de passe avec l'algorithme Argon2id.

III. Périmètre Fonctionnel

III.1 Exigences Fonctionnelles

L'application se concentre sur les besoins "métier" de l'employé :

1. **Authentification Sécurisée** : Accès restreint au personnel via un identifiant et mot de passe chiffré.
2. **Gestion du Catalogue (CRUD)** :
   - ○ Ajout de livres par scan/saisie d'ISBN (récupération auto des métadonnées : titre, auteur, résumé, couverture).
   - ○ Gestion manuelle des exemplaires (état du livre, disponibilité).
3. **Gestion des Adhérents** : Création de fiches lecteurs (nom, coordonnées, date d'adhésion).
4. **Module de Prêts/Retours** :
   - ○ Enregistrement d'un prêt en associant un livre à un adhérent.
   - ○ Calcul automatique de la date de retour prévue.
   - ○ Validation du retour et mise à jour immédiate du stock.
5. **Tableau de Bord de Suivi** : Visualisation des livres en retard et statistiques simples (livres les plus loués).

Hors périmètre : Réservation en ligne par les usagers (l'application est strictement interne).

III.2 Exigences Non Fonctionnelles

Ces exigences définissent les qualités attendues du système, indépendamment des fonctionnalités :

Performance	Ajout d'un livre via ISBN	≤ 5 secondes (appel API + affichage formulaire pré-rempli)
Sécurité	Protection contre le Top 10 OWASP	Injections SQL bloquées (Doctrine ORM), XSS (React), CSRF (API stateless JWT), hachage Argon2id

Conformité	RGPD	Registre de traitement documenté, droit à l'oubli implémenté avant mise en production
Maintenabilité	Couverture de tests	Pipeline CI/CD avec 100 % des tests passants à chaque build
Compatibilité	Systèmes d'exploitation	Windows 10+ et macOS 12+ (installateur natif Tauri)
Disponibilité	Mode hors connexion partiel	Consultation du catalogue et des prêts possible sans connexion internet active

IV. Exigences Techniques

Pour répondre aux besoins de performance et de maintenabilité, les choix suivants ont été arrêtés :

- **Architecture** : Approche **Découplée (Headless)**. Une API Symfony pour la logique métier et une application desktop React (via **Tauri**) pour l'interface. Ce choix garantit une expérience utilisateur fluide (UX native) tout en isolant la logique de données.
- **Stack Technique** :
  - **Backend** : PHP 8.3+, Symfony 7 (API Platform).
  - **Frontend** : React 18, Tailwind CSS, Tauri (pour l'installateur Desktop).
  - **Base de Données** : PostgreSQL (hébergé sur Supabase).
- **Infrastructure & Qualité** :
  - **Conteneurisation** : Docker pour l'homogénéité des environnements.
  - **CI/CD** : Pipeline via GitHub Actions (Tests automatisés et build des exécutables).
  - **Tests** : Tests unitaires (PHPUnit), tests d'intégration et tests fonctionnels.
- **Sécurité** : Protection contre le Top 10 OWASP (Injections SQL via Doctrine ORM, XSS via React, CSRF via Symfony, hachage Argon2id).

V. Contraintes et Enjeux du Projet

1. Calendrier et Jalons

Le projet s'étale sur 6 mois (Janvier - Juin 2026) :

- **Janvier** : CDCF (Jalon actuel).
- **Février** : Méthodologie de Projet & Conception UI/UX.
- **Mars** : Modélisation de la Base de Données.

● **Avril** : Conception de l’application & Architecture.
● **Mai** : Développement, Sécurité & Tests (version Bêta).
● **Juin** : Déploiement et Mise en Production (Livrable final).

2. Risques Identifiés

```
● API Tierce : Dépendance aux API de recherche de livres (ex: Open Library).
Solution : Prévoir un mode de saisie manuelle de secours.
● Sécurité des données : Manipulation de données personnelles (Adhérents).
Solution : Conformité stricte RGPD (chiffrement, droit à l'oubli, registre de
traitement).
● Délai : Périmètre fonctionnel large pour une personne seule. Solution : Priorisation
stricte du MVP (Prêts/Retours) avant les statistiques.
```

3. Critères de Succès

Le projet sera considéré comme réussi si :

6. L'installateur (.exe ou .msi) est fonctionnel et s'interface correctement avec l'API.

7. Un livre peut être ajouté en moins de 5 secondes via son ISBN.
8. Le pipeline CI/CD valide 100% des tests à chaque déploiement.
9. L'interface est jugée intuitive par un utilisateur test lors de la démonstration finale.

VI. Glossaire

API	Application Programming Interface – interface permettant à deux logiciels de communiquer via des requêtes HTTP normalisées.
CI/CD	Continuous Integration / Continuous Deployment – automatisation des tests et du déploiement à chaque modification du code source (ici via GitHub Actions).
CRUD	Create, Read, Update, Delete – les quatre opérations de base sur une base de données.
Docker	Outil de conteneurisation permettant d'exécuter des applications dans des environnements isolés et reproductibles.
Headless	Architecture découplée : le backend (API) et le frontend (interface) sont deux applications distinctes communiquant via HTTP.

ISBN	International Standard Book Number – identifiant numérique unique attribué à chaque édition d'un livre (13 chiffres).
JWT	JSON Web Token – format de jeton d'authentification signé, utilisé pour sécuriser les échanges entre le frontend et l'API.
MVP	Minimum Viable Product – version minimale du produit incluant uniquement les fonctionnalités essentielles pour valider le projet.
OWASP	Open Web Application Security Project – organisation référençant les 10 failles de sécurité les plus critiques des applications web.
ORM	Object-Relational Mapping – couche d'abstraction permettant de manipuler la base de données via des objets (ici : Doctrine pour Symfony).
RGPD	Règlement Général sur la Protection des Données – réglementation européenne encadrant la collecte et le traitement des données personnelles.
SIGB	Système Intégré de Gestion de Bibliothèque – logiciel métier complet utilisé par les grandes bibliothèques (ex : PMB, Koha).
Tauri	Framework open source permettant de créer des applications desktop natives (Windows, macOS, Linux) à partir d'un frontend web (React).

Jalon 2 – Méthodologie & Conception UI/UX

Projet : BiblioTech Desktop
Auteur : Grégory Sergent
Formation : CDA – Concepteur Développeur d'Applications
Période : Janvier → Juin 2026
Date : 28/02/2026

Partie 1 – Méthodologie et organisation du projet

1.1 Méthode de gestion

Approche choisie : Agile – Scrum adapté en solo.

- **Cadence :** 1 sprint par mois (aligné sur les jalons).
- **Définition :** un *sprint* est une période courte et fixe durant laquelle on réalise un lot d'objectifs précis.
- **Rituels adaptés :**

- **Sprint planning mensuel** : définition des objectifs du jalon + découpage en tâches.
- **Revue hebdomadaire** : contrôle de l'avancement, mise à jour du backlog Trello.
- **Rétrospective mensuelle** : bilan des difficultés/solutions et ajustements.

- **Justification :** ce format léger garde un cadre clair tout en restant réaliste pour un travail en solo.

1.2 Macro-planning (jusqu'à juin 2026)

- **Janvier** : Cadrage fonctionnel – livrable : CDCF (Jalon 1)
- **Février** : Organisation & design UI/UX – livrable : Méthodologie + maquettes (Jalon 2)
- **Mars** : Conception des données – livrable : MCD/MLD/MPD + dictionnaire (Jalon 3)
- **Avril** : Conception applicative & architecture – livrable : UML + archi (Jalon 4)
- **Mai** : Implémentation + sécurité + tests – livrable : Version bêta + preuves CI + tests (Jalon 5)
- **Juin** : Déploiement & finalisation – livrable : Livraison finale + mise en prod (Jalon 6)

1.3 Suivi des tâches

Outil : Trello (Kanban)

- Colonnes : *Backlog* → *À faire* → *En cours* → *En revue* → *Terminé*
- **Rythme :** mise à jour hebdomadaire, ajustement du backlog à chaque sprint planning.
- **Traçabilité :** chaque carte correspond à une user story ou tâche technique, avec checklist si besoin.

1.4 Gestion du code source (Git)

- **Branches :**

- `main` : versions stables (jalons)
- `develop` : intégration continue
- `feature/*` : nouvelles fonctionnalités ou correctifs

- **Process :** PR systématique vers `develop` (auto-revue avant merge).
- **Conventions :** commits courts et explicites (ex. `feat: isbn lookup`, `fix: loan return status`).

1.5 CI/CD planifié

CI (objectif si le temps le permet)

- **Cible : GitHub Actions**
- À chaque push (objectif) :

- Tests PHP (PHPUnit) pour l'API Symfony
- Lint/format (PHP-CS-Fixer / ESLint)
- Build front React
- Build Docker (API)

- **Plan B si manque de temps :** exécuter les tests et le build localement avant chaque jalon, avec checklist documentée.

CD (à partir de mai/juin)

- Publication d'images Docker (API) vers Docker Hub ou GHCR
- Build desktop Tauri (Windows) via pipeline CI ou en local si nécessaire
- Livraison du binaire `*.exe` en artefact release

Partie 2 – Conception UI/UX

2.1 Sitemap (écrans principaux)

Le sitemap ci-dessous présente l'ensemble des écrans de l'application BiblioTech Desktop et les liens de navigation entre eux.

- **Authentification**

- Connexion

- **Tableau de bord**

- Vue synthétique (prêts en retard + statistiques)

- **Catalogue**

- Liste des livres
- Détail d'un livre
- Ajout / Édition d'un livre

- **Adhérents**

- Liste des adhérents
- Détail d'un adhérent
- Ajout / Édition d'un adhérent

- **Prêts / Retours**

- Création d'un prêt
- Liste des prêts (tableau + barre de recherche)
- Retour d'un livre

2.2 Zoning (écran type)

Desktop – Écran "Authentification"

- Zone centrale : Formulaire de connexion (identifiant + mot de passe)
- Zone inférieure / action : Bouton de connexion

Desktop – Écran "Tableau de bord"

- Navbar : Navigation principale
- Zone statistiques : Cartes de statistiques de prêt (nb prêts en cours, retards, etc.)
- Zone tableau 1 : Tableau des prêts en retard
- Zone tableau 2 : Tableau des derniers prêts

Desktop – Écran "Catalogue"

- Navbar : Navigation principale
- Zone recherche : Barre de recherche
- Zone tableau : Liste des livres

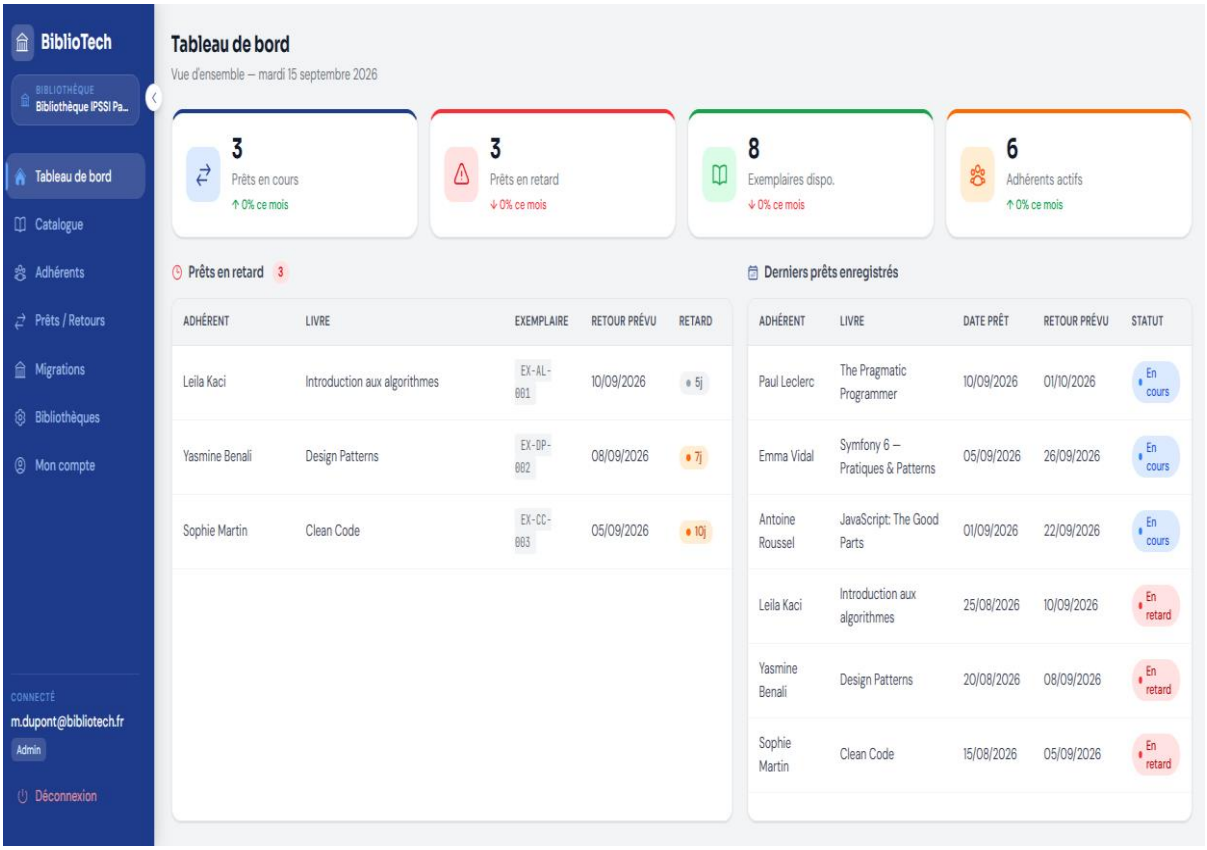
- Zone détail : Détails du livre sélectionné
- Zone actions : Boutons d'ajout et d'édition de livre

Desktop – Écran "Adhérents"

- Navbar : Navigation principale
- Zone recherche : Barre de recherche
- Zone liste : Liste des adhérents
- Zone détail : Détails de l'adhérent sélectionné
- Zone actions : Boutons d'ajout, d'édition et de suppression d'adhérent

Desktop – Écran "Prêts / Retours"

- Navbar : Navigation principale
- Zone recherche : Barre de recherche
- Zone liste : Liste des prêts en cours
- Zone détail : Détails du prêt sélectionné
- Zone actions : Boutons "Créer un prêt" et "Rendre un prêt"



2.3 Wireframes (basse fidélité)

A) Desktop – Authentification

- Logo de l'application (haut droite)
- 2 labels (identifiant, mot de passe)
- 2 champs de saisie
- 1 bouton de connexion

B) Desktop – Tableau de bord

- Navbar gauche : liste de navigation + bouton déconnexion
- Zone stats (haut) : 3 blocs de statistiques
- 2 tableaux liste en dessous (prêts en retard + derniers prêts)

C) Desktop – Catalogue

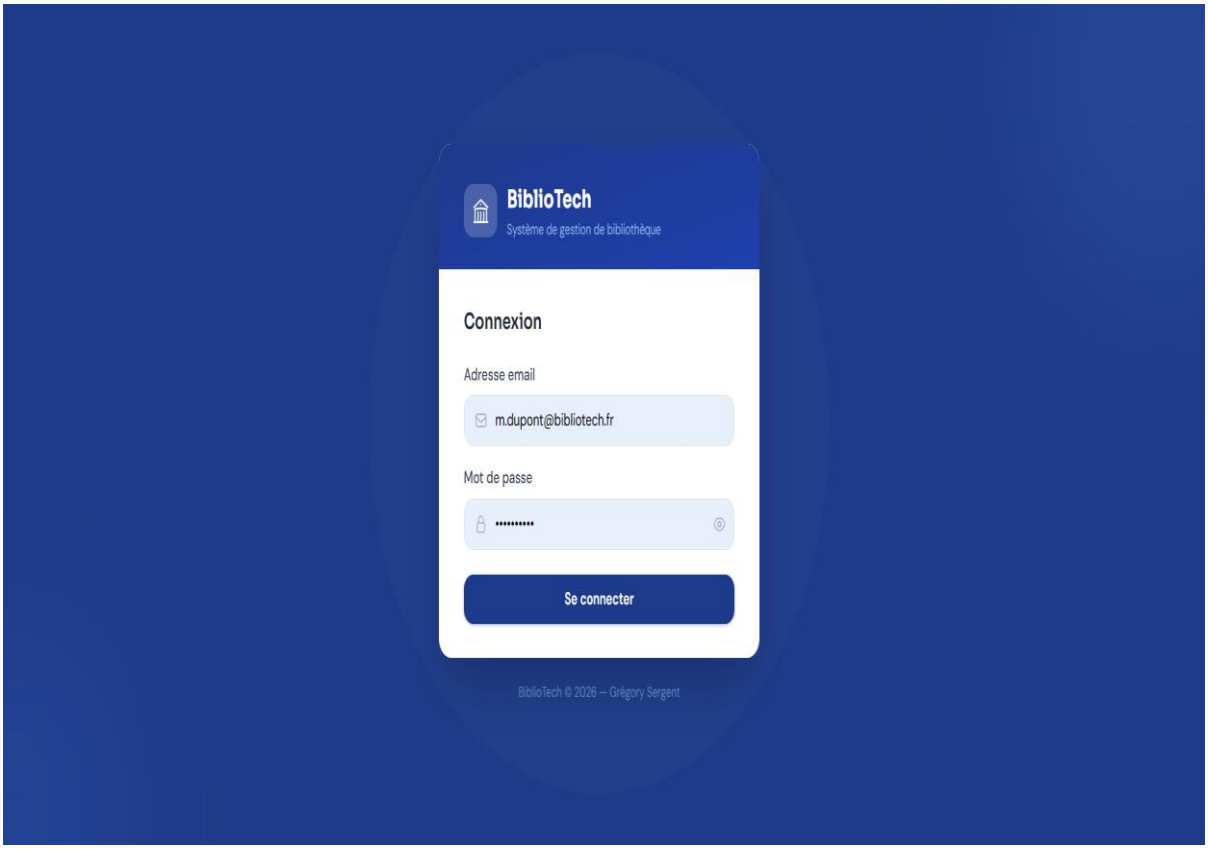
- Navbar gauche : liste de navigation + bouton déconnexion
- Barre de recherche (haut de l'écran)
- Tableau liste des livres
- Zone détail du livre sélectionné (à droite du tableau)
- 2 boutons sous le tableau : édition + ajout d'un livre

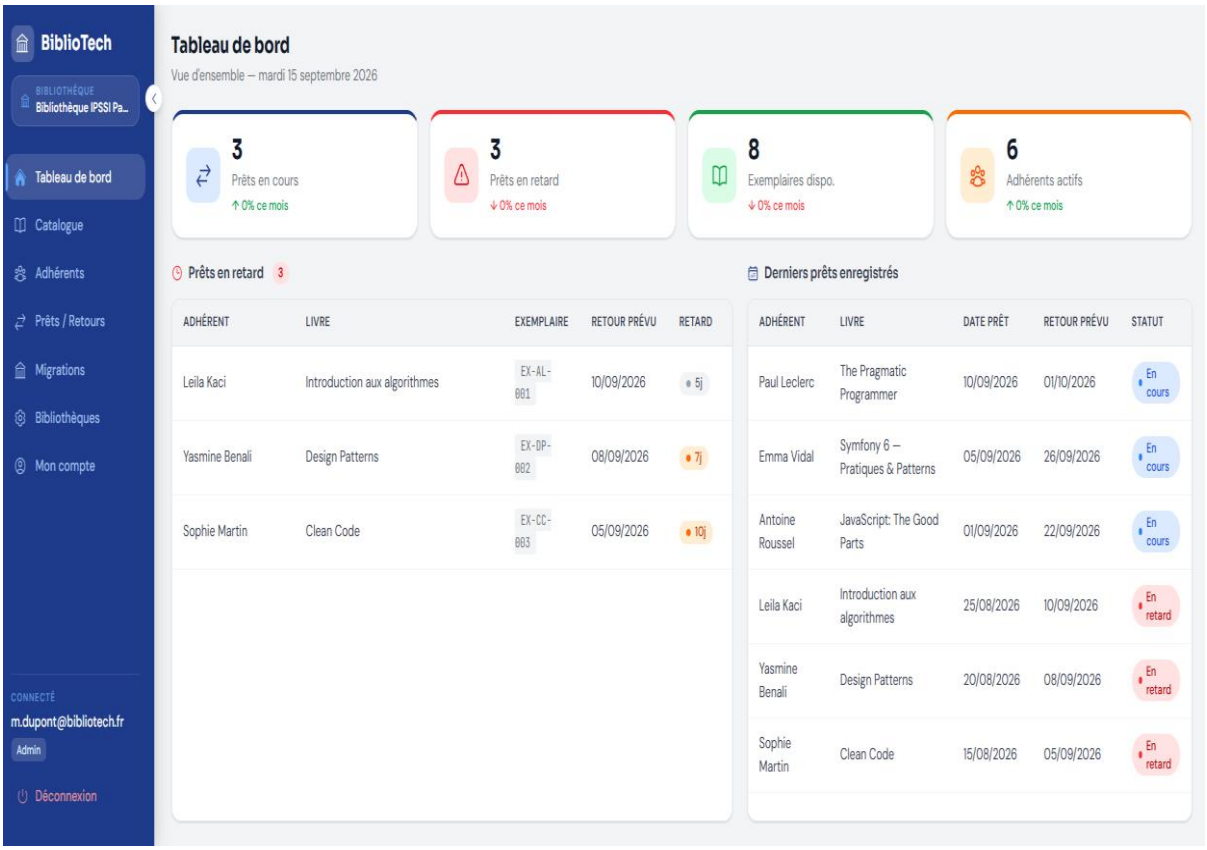
D) Desktop – Adhérents

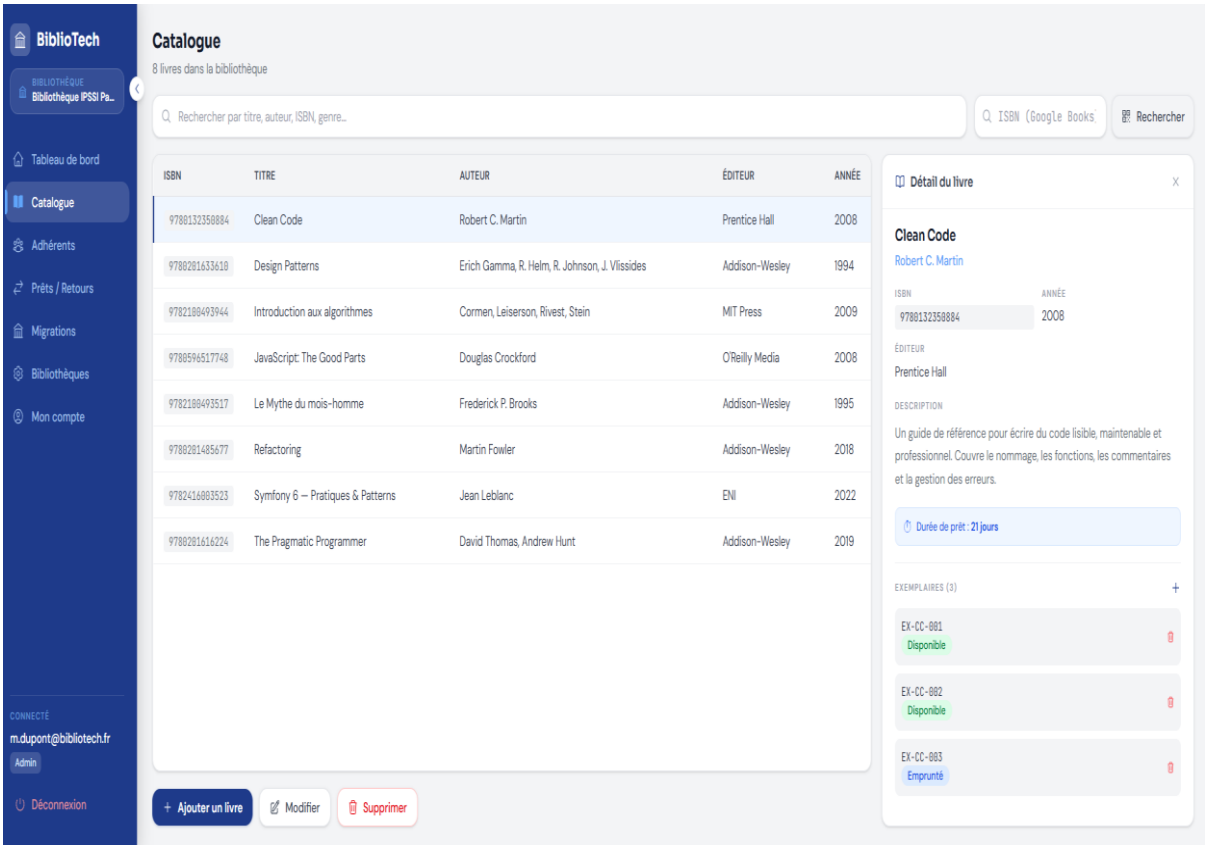
- Navbar gauche : liste de navigation + bouton déconnexion
- Barre de recherche (haut de l'écran)
- Tableau liste des adhérents
- Zone détail de l'adhérent sélectionné (à droite du tableau)
- 3 boutons sous le tableau : ajout + édition + suppression d'un adhérent

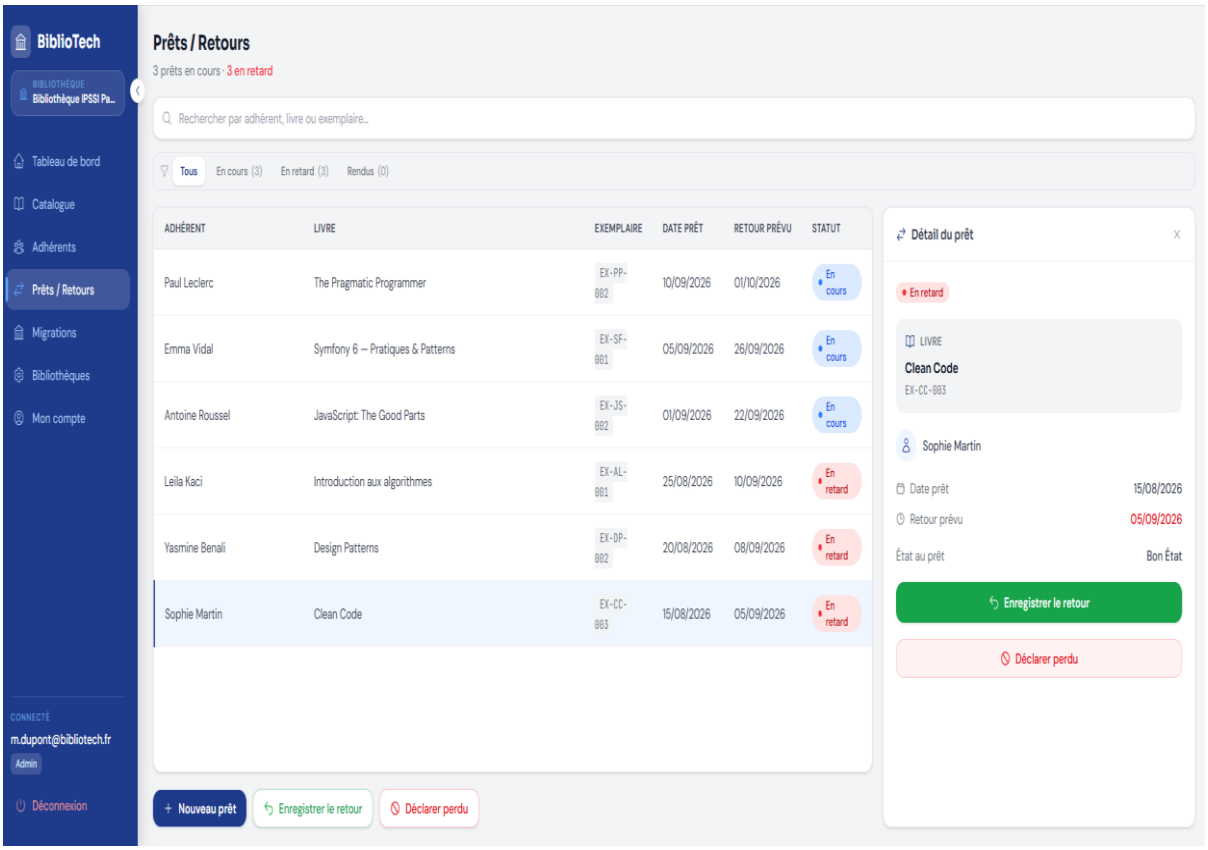
E) Desktop – Prêts / Retours

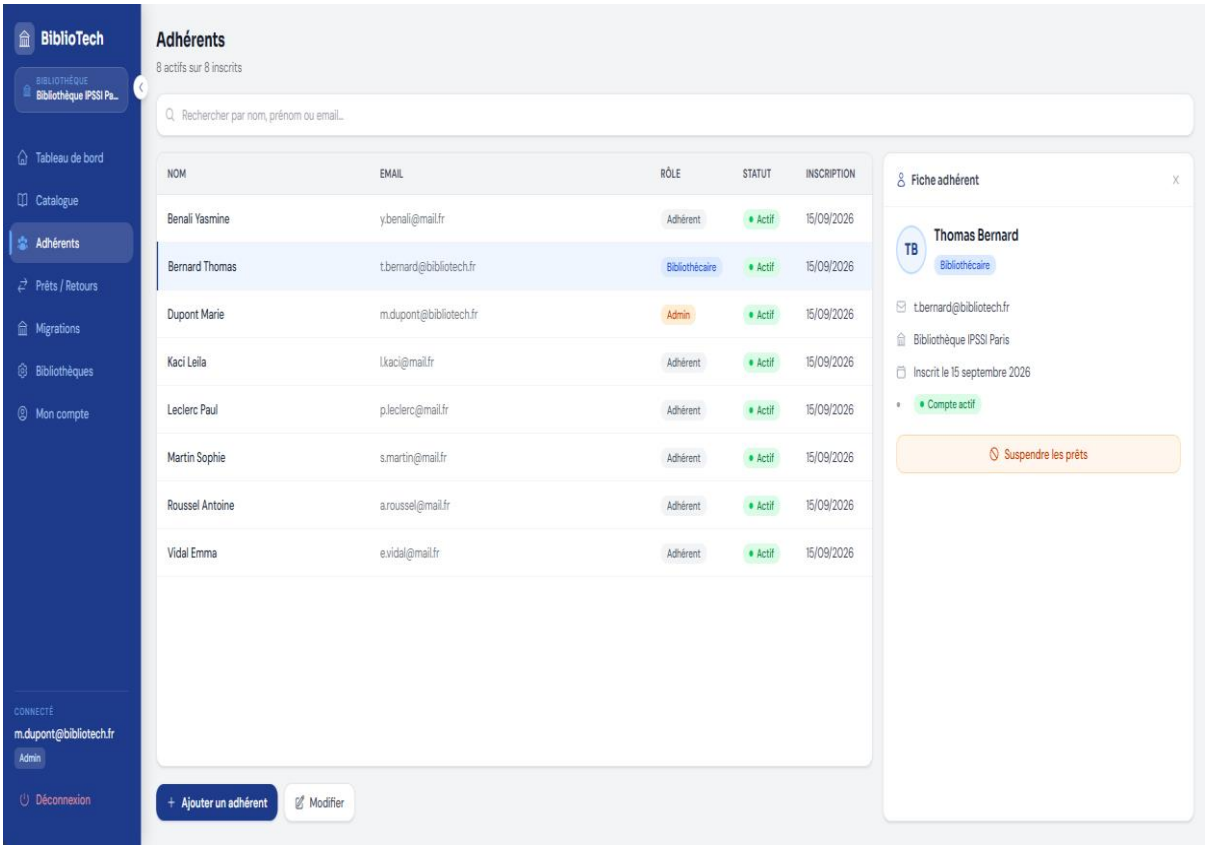
- Navbar gauche : liste de navigation + bouton déconnexion
- Barre de recherche (haut de l'écran)
- Tableau liste des prêts en cours
- Zone détail du prêt sélectionné (à droite du tableau)
- 2 boutons sous le tableau : créer un prêt + rendre un prêt


BiblioTech
Système de gestion de bibliothèque
Connexion
Adresse email
m.dupont@bibliotech.fr
Mot de passe
Se connecter
BiblioTech © 2026 – Grégory Sergent


BiblioTech
BIBLIOTHÈQUE
Bibliothèque IPSSI Pa...
Tableau de bord
Catalogue
Adhérents
Prêts / Retours
Migrations
Bibliothèques
Mon compte
CONNECTÉ
m.dupont@bibliotech.fr
Admin
Déconnexion
Tableau de bord
Vue d'ensemble – mardi 15 septembre 2026
3
Prêts en cours
↑ 0% ce mois
3
Prêts en retard
↓ 0% ce mois
8
Exemplaires dispo.
↓ 0% ce mois
6
Adhérents actifs
↑ 0% ce mois
Prêts en retard 3
ADHÉRENT LIVRE EXEMPLAIRE RETOUR PRÉVU RETARD
Leila Kaci Introduction aux algorithmes EX-AL-001 10/09/2026 5j
Yasmine Benali Design Patterns EX-DP-002 08/09/2026 7j
Sophie Martin Clean Code EX-CC-003 05/09/2026 10j
Derniers prêts enregistrés
ADHÉRENT LIVRE DATE PRÊT RETOUR PRÉVU STATUT
Paul Leclerc The Pragmatic Programmer 10/09/2026 01/10/2026 En cours
Emma Vidal Symfony 6 – Pratiques & Patterns 05/09/2026 26/09/2026 En cours
Antoine Roussel JavaScript: The Good Parts 01/09/2026 22/09/2026 En cours
Leila Kaci Introduction aux algorithmes 25/08/2026 10/09/2026 En retard
Yasmine Benali Design Patterns 20/08/2026 08/09/2026 En retard
Sophie Martin Clean Code 15/08/2026 05/09/2026 En retard


BiblioTech
BIBLIOTHÈQUE
Bibliothèque IPSSI Pa...
Tableau de bord
Catalogue
Adhérents
Prêts / Retours
Migrations
Bibliothèques
Mon compte
CONNECTÉ
m.dupont@bibliotech.fr
Admin
Déconnexion
Catalogue
8 livres dans la bibliothèque
Rechercher par titre, auteur, ISBN, genre...
ISBN (Google Books)
Rechercher
ISBN
TITRE
AUTEUR
ÉDITEUR
ANNÉE
9780132350884
Clean Code
Robert C. Martin
Prentice Hall
2008
9780201633610
Design Patterns
Erich Gamma, R. Helm, R. Johnson, J. Vlissides
Addison-Wesley
1994
9782180493944
Introduction aux algorithmes
Cormen, Leiserson, Rivest, Stein
MIT Press
2009
9780596517748
JavaScript: The Good Parts
Douglas Crockford
O'Reilly Media
2008
9782180493517
Le Mythe du mois-homme
Frederick P. Brooks
Addison-Wesley
1995
9780201485677
Refactoring
Martin Fowler
Addison-Wesley
2018
9782416083523
Symfony 6 – Pratiques & Patterns
Jean Leblanc
ENI
2022
9780201616224
The Pragmatic Programmer
David Thomas, Andrew Hunt
Addison-Wesley
2019
Détail du livre
Clean Code
Robert C. Martin
ISBN
9780132350884
ANNÉE
2008
ÉDITEUR
Prentice Hall
DESCRIPTION
Un guide de référence pour écrire du code lisible, maintenable et professionnel. Couvre le nommage, les fonctions, les commentaires et la gestion des erreurs.
Durée de prêt : 21 jours
EXEMPLAIRES (3)
EX-CC-001
Disponible
EX-CC-002
Disponible
EX-CC-003
Emprunté
Ajouter un livre
Modifier
Supprimer


BiblioTech
BIBLIOTHÈQUE
Bibliothèque IPSSI Pa...
Tableau de bord
Catalogue
Adhérents
Prêts / Retours
Migrations
Bibliothèques
Mon compte
CONNECTÉ
m.dupont@bibliotech.fr
Admin
Déconnexion
Prêts / Retours
3 prêts en cours · 3 en retard
Rechercher par adhérent, livre ou exemplaire...
Tous
En cours (3)
En retard (3)
Rendus (0)
ADHÉRENT
LIVRE
EXEMPLAIRE
DATE PRÊT
RETOUR PRÉVU
STATUT
Paul Leclerc
The Pragmatic Programmer
EX-PP-002
10/09/2026
01/10/2026
En cours
Emma Vidal
Symfony 6 – Pratiques & Patterns
EX-SF-001
05/09/2026
26/09/2026
En cours
Antoine Roussel
JavaScript: The Good Parts
EX-JS-002
01/09/2026
22/09/2026
En cours
Leila Kaci
Introduction aux algorithmes
EX-AL-001
25/08/2026
10/09/2026
En retard
Yasmine Benali
Design Patterns
EX-DP-002
20/08/2026
08/09/2026
En retard
Sophie Martin
Clean Code
EX-CC-003
15/08/2026
05/09/2026
En retard
Détail du prêt
En retard
LIVRE
Clean Code
EX-CC-003
Sophie Martin
Date prêt
15/08/2026
Retour prévu
05/09/2026
État au prêt
Bon État
Enregistrer le retour
Déclarer perdu
Nouveau prêt
Enregistrer le retour
Déclarer perdu



2.4 Charte graphique

Palette de couleurs

- Primaire : Bleu nuit – `#1E3A8A`
- Secondaire : Bleu clair – `#60A5FA`
- Accent / Validation : Vert – `#16A34A`
- Neutre clair : Gris clair – `#F3F4F6`
- Neutre foncé : Gris foncé – `#374151`

Justification : le bleu nuit inspire confiance et sérieux, adapté à un outil professionnel interne. Le vert est réservé aux actions de validation pour un feedback visuel immédiat.

Typographie

- Titre principal (H1) : **Sora** – Bold – 28px
- Titres de section (H2/H3) : **Sora** – SemiBold – 20px / 16px
- Texte courant : **DM Sans** – Regular – 14px
- Labels / libellés : **DM Sans** – Medium – 13px

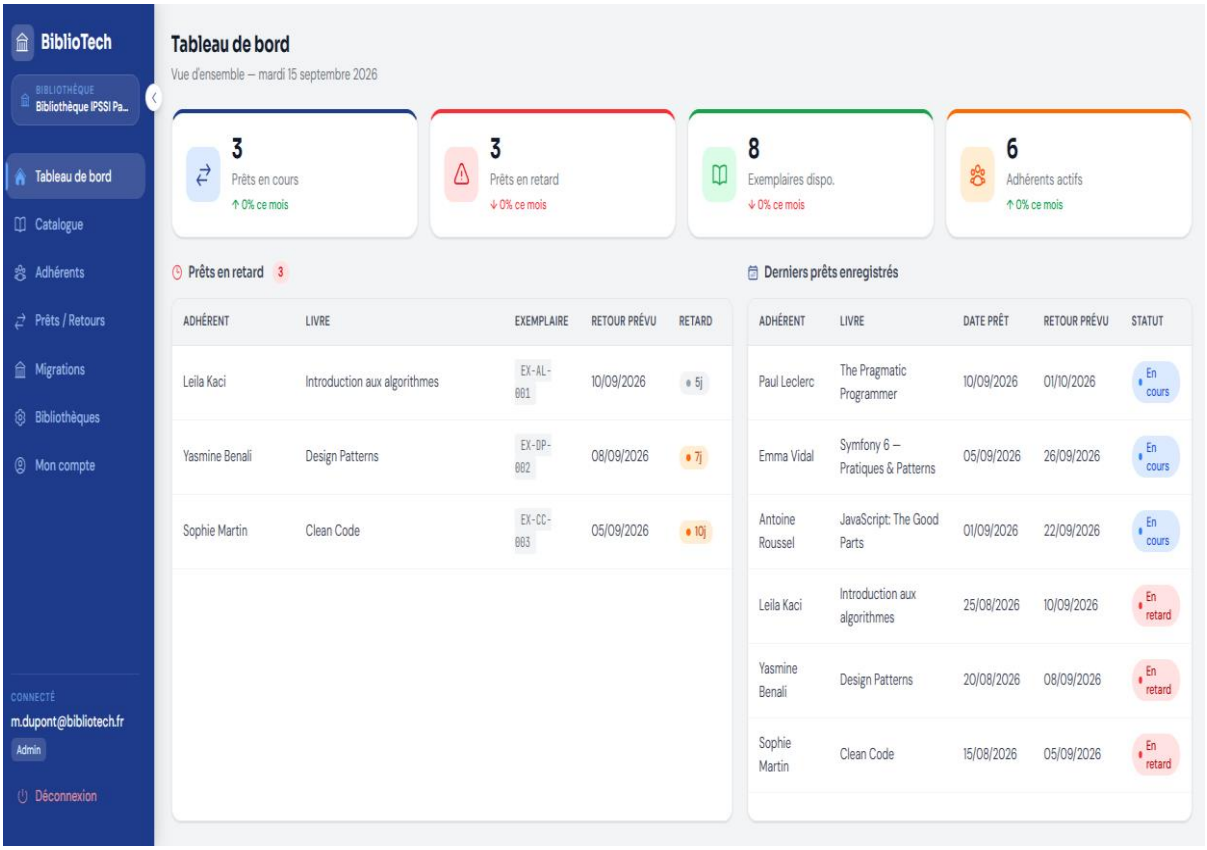
- Légendes / métadonnées : **DM Sans** – Regular – 12px
- Code / ISBN : **JetBrains Mono** – Regular – 13px

Justification :

- **Sora** – police géométrique moderne pour les titres, contemporaine et professionnelle.
- **DM Sans** – conçue pour les interfaces numériques, très lisible à petite taille, idéale pour tableaux et formulaires.
- **JetBrains Mono** – réservée aux identifiants techniques (ISBN, codes) pour les distinguer du reste du contenu.

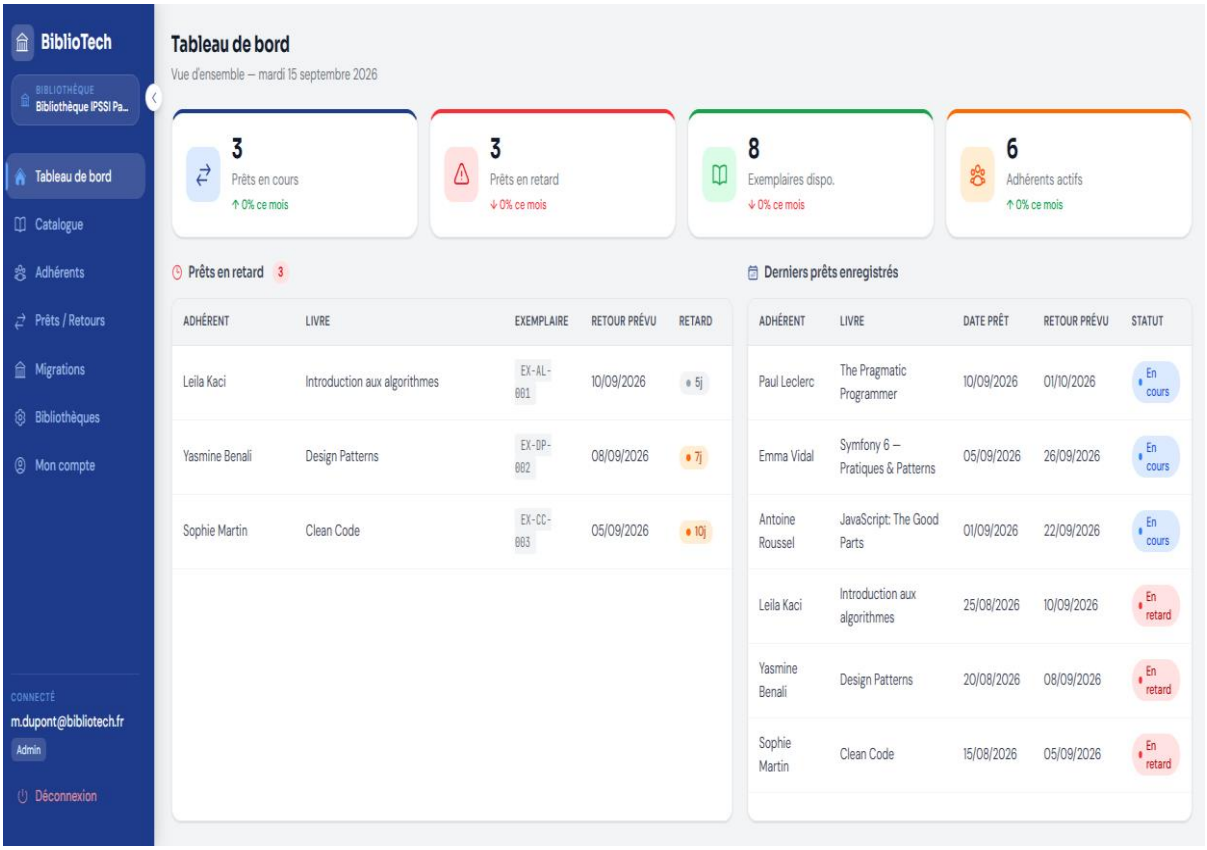
Style général

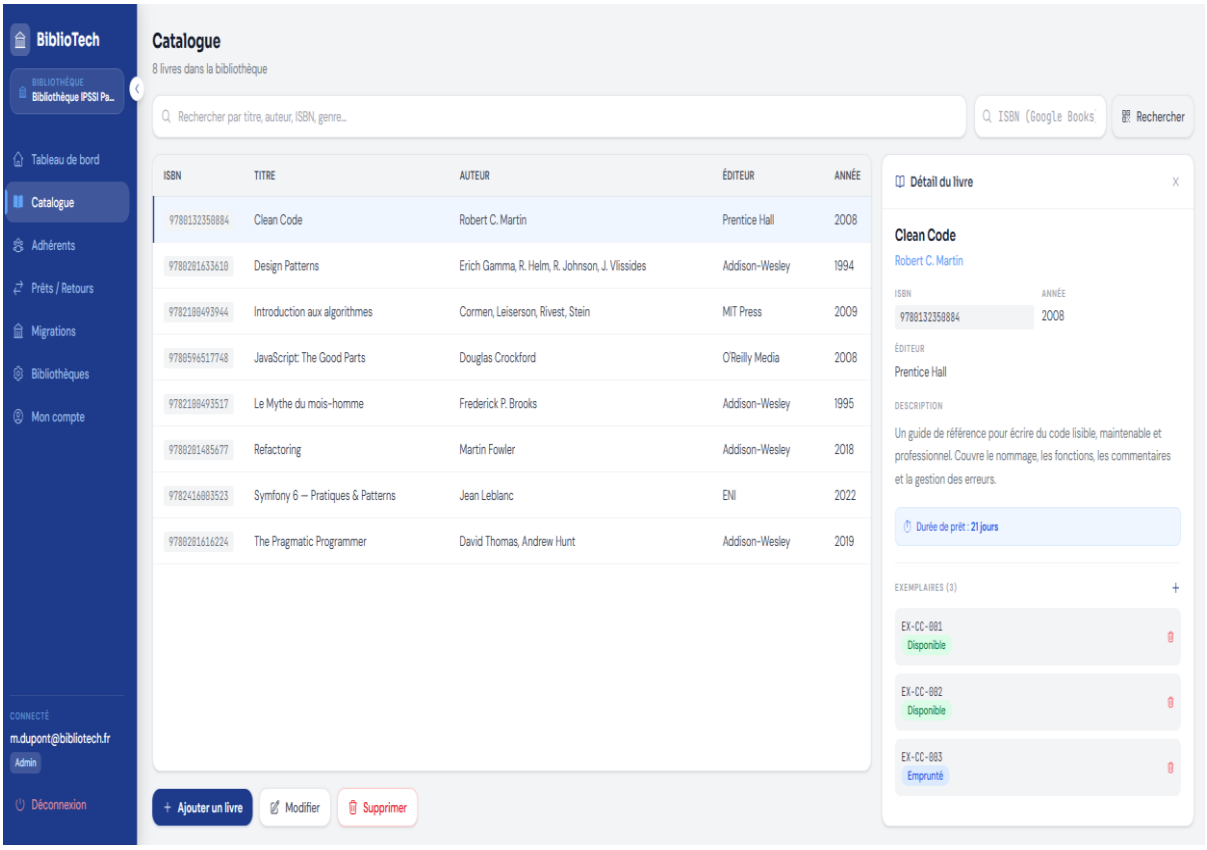
- Icônes linéaires : **Heroicons**
- Espacements généreux pour maximiser la lisibilité
- Interface sobre et professionnelle, sans effets décoratifs superflus

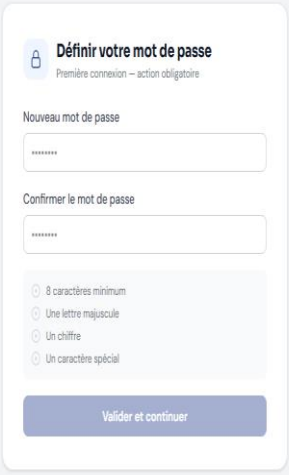


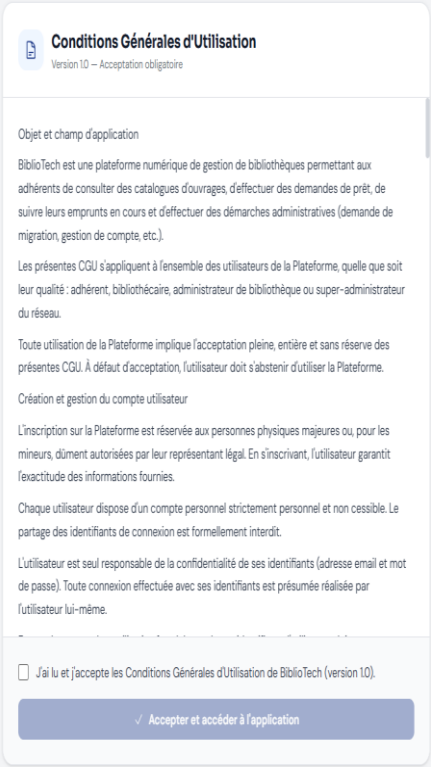
2.5 Maquettes graphiques (haute fidélité)

Version Desktop – Tableau de bord

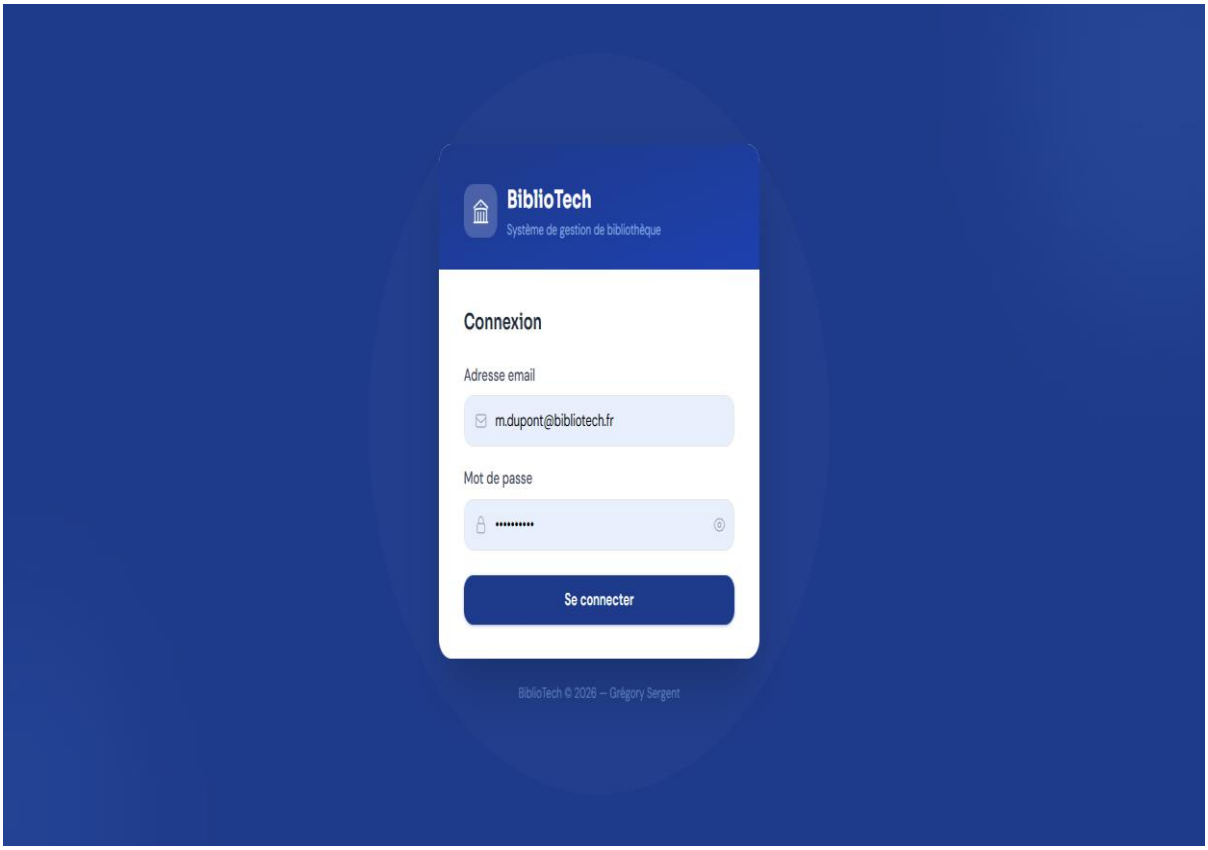

BiblioTech
BIBLIOTHÈQUE
Bibliothèque IPSSI Pa...
Tableau de bord
Catalogue
Adhérents
Prêts / Retours
Migrations
Bibliothèques
Mon compte
CONNECTÉ
m.dupont@bibliotech.fr
Admin
Déconnexion
Tableau de bord
Vue d'ensemble – mardi 15 septembre 2026
3
Prêts en cours
↑ 0% ce mois
3
Prêts en retard
↓ 0% ce mois
8
Exemplaires dispo.
↓ 0% ce mois
6
Adhérents actifs
↑ 0% ce mois
Prêts en retard 3
ADHÉRENT LIVRE EXEMPLAIRE RETOUR PRÉVU RETARD
Leila Kaci Introduction aux algorithmes EX-AL-001 10/09/2026 5j
Yasmine Benali Design Patterns EX-DP-002 08/09/2026 7j
Sophie Martin Clean Code EX-CC-003 05/09/2026 10j
Derniers prêts enregistrés
ADHÉRENT LIVRE DATE PRÊT RETOUR PRÉVU STATUT
Paul Leclerc The Pragmatic Programmer 10/09/2026 01/10/2026 En cours
Emma Vidal Symfony 6 – Pratiques & Patterns 05/09/2026 26/09/2026 En cours
Antoine Roussel JavaScript: The Good Parts 01/09/2026 22/09/2026 En cours
Leila Kaci Introduction aux algorithmes 25/08/2026 10/09/2026 En retard
Yasmine Benali Design Patterns 20/08/2026 08/09/2026 En retard
Sophie Martin Clean Code 15/08/2026 05/09/2026 En retard


BiblioTech
BIBLIOTHÈQUE
Bibliothèque IPSSI Pa...
Tableau de bord
Catalogue
Adhérents
Prêts / Retours
Migrations
Bibliothèques
Mon compte
CONNECTÉ
m.dupont@bibliotech.fr
Admin
Déconnexion
Catalogue
8 livres dans la bibliothèque
Rechercher par titre, auteur, ISBN, genre...
ISBN (Google Books)
Rechercher
ISBN TITRE AUTEUR ÉDITEUR ANNÉE
9780132350884 Clean Code Robert C. Martin Prentice Hall 2008
9780201633610 Design Patterns Erich Gamma, R. Helm, R. Johnson, J. Vlissides Addison-Wesley 1994
9782100493944 Introduction aux algorithmes Cormen, Leiserson, Rivest, Stein MIT Press 2009
9780596517748 JavaScript: The Good Parts Douglas Crockford O'Reilly Media 2008
9782100493517 Le Mythe du mois-homme Frederick P. Brooks Addison-Wesley 1995
9780201485677 Refactoring Martin Fowler Addison-Wesley 2018
9782416003523 Symfony 6 – Pratiques & Patterns Jean Leblanc ENI 2022
9780201616224 The Pragmatic Programmer David Thomas, Andrew Hunt Addison-Wesley 2019
Détail du livre
Clean Code
Robert C. Martin
ISBN 9780132350884
ANNÉE 2008
ÉDITEUR Prentice Hall
DESCRIPTION
Un guide de référence pour écrire du code lisible, maintenable et professionnel. Couvre le nommage, les fonctions, les commentaires et la gestion des erreurs.
Durée de prêt : 21 jours
EXEMPLAIRES (3)
EX-CC-001 Disponible
EX-CC-002 Disponible
EX-CC-003 Emprunté
Ajouter un livre
Modifier
Supprimer


Définir votre mot de passe
Première connexion — action obligatoire
Nouveau mot de passe
Confirmer le mot de passe
8 caractères minimum
Une lettre majuscule
Un chiffre
Un caractère spécial
Valider et continuer


Conditions Générales d'Utilisation
Version 1.0 — Acceptation obligatoire
Objet et champ d'application
BiblioTech est une plateforme numérique de gestion de bibliothèques permettant aux adhérents de consulter des catalogues d'ouvrages, d'effectuer des demandes de prêt, de suivre leurs emprunts en cours et d'effectuer des démarches administratives (demande de migration, gestion de compte, etc.).
Les présentes CGU s'appliquent à l'ensemble des utilisateurs de la Plateforme, quelle que soit leur qualité : adhérent, bibliothécaire, administrateur de bibliothèque ou super-administrateur du réseau.
Toute utilisation de la Plateforme implique l'acceptation pleine, entière et sans réserve des présentes CGU. À défaut d'acceptation, l'utilisateur doit s'abstenir d'utiliser la Plateforme.
Création et gestion du compte utilisateur
L'inscription sur la Plateforme est réservée aux personnes physiques majeures ou, pour les mineurs, dûment autorisées par leur représentant légal. En s'inscrivant, l'utilisateur garantit l'exactitude des informations fournies.
Chaque utilisateur dispose d'un compte personnel strictement personnel et non cessible. Le partage des identifiants de connexion est formellement interdit.
L'utilisateur est seul responsable de la confidentialité de ses identifiants (adresse email et mot de passe). Toute connexion effectuée avec ses identifiants est présumée réalisée par l'utilisateur lui-même.
J'ai lu et j'accepte les Conditions Générales d'Utilisation de BiblioTech (version 1.0).
√ Accepter et accéder à l'application

Version Mobile – Liste du catalogue



2.6 Considérations UX

- **Scan ISBN :** ajout d'un livre en moins de 5 secondes via scan ou saisie manuelle.
- **Feedback visuel :** messages de confirmation/erreur clairs sur chaque action critique (prêt validé, ISBN invalide, adhérent introuvable).
- **Accessibilité :** contrastes élevés conformes aux recommandations WCAG, navigation clavier complète.
- **Performance :** listes paginées pour éviter les temps de chargement excessifs même avec un grand catalogue.
- **Mobile First :** l'interface s'adapte à des résolutions réduites, avec des zones tactiles suffisamment grandes.

Jalon 3 – Modélisation de la Base de Données

Projet : BiblioTech

Auteur : Grégory Sergent

Formation : CDA – Concepteur Développeur d'Applications

Période : Janvier → Juin 2026

Date : 31/03/2026

---

0) Évolution du périmètre fonctionnel

La phase de modélisation (Jalon 3) a conduit à affiner le périmètre défini dans le CDCF (Jalon 1). Les évolutions suivantes sont actées :

Ajout : comptes adhérents

Le Jalon 1 positionnait BiblioTech comme une application strictement interne au personnel. La conception de la base de données a mis en évidence la nécessité d'offrir aux adhérents un accès limité en lecture à l'application, afin de leur permettre de :

- Consulter le **catalogue et le stock** de leur bibliothèque (lecture seule, filtré par `bibliotheque_id`)
- Consulter leurs **prêts en cours** et leur historique de prêts
- Soumettre une **demande de migration** vers une autre bibliothèque

Ces actions n'impliquent aucune réservation en ligne ni modification de données — elles restent en lecture seule ou via un workflow validé par un admin. La contrainte "hors périmètre : réservation en ligne" du Jalon 1 est donc toujours respectée.

Ce que les adhérents ne peuvent PAS faire :

- Réserver un exemplaire
- Modifier leurs informations personnelles (géré par l'admin/bibliothécaire)
- Accéder au catalogue d'autres bibliothèques
- Accéder à toute fonctionnalité d'administration

Cette évolution est prise en compte dans le dictionnaire des données (rôle `adherent` dans `utilisateurs`) et dans la table `demandes_migration`.

---

1) Introduction – Démarche MERISE

La modélisation de la base de données suit la méthode **MERISE**, en trois niveaux progressifs :

- **MCD** (Modèle Conceptuel de Données) : représente les entités métier et leurs relations, sans considération technique.
- **MLD** (Modèle Logique de Données) : traduit le MCD en tables relationnelles avec clés primaires et étrangères.
- **MPD** (Modèle Physique de Données) : décline le MLD avec les types SQL concrets et les contraintes de la base cible (PostgreSQL).

Cette démarche garantit une conception rigoureuse, indépendante du SGBD dans un premier temps, avant d'être adaptée à PostgreSQL via Supabase.

---

2) Dictionnaire des données

Le dictionnaire des données liste l'ensemble des attributs de chaque entité, leur type générique, leur rôle et leurs contraintes.

2.1 Entité : bibliotheques

Représente une bibliothèque cliente de la plateforme.

- **id** – Entier – Identifiant unique – Clé primaire, auto-incrémenté
- **nom** – Texte – Nom de la bibliothèque – Obligatoire
- **adresse** – Texte – Adresse postale complète – Obligatoire
- **email_contact** – Texte – Email de contact principal – Obligatoire, unique
- **telephone** – Texte – Numéro de téléphone – Optionnel
- **date_inscription** – Date – Date d'enregistrement sur la plateforme – Obligatoire, générée automatiquement
- **actif** – Booléen – Indique si la bibliothèque est active – Par défaut : vrai

2.2 Entité : utilisateurs

Représente tous les comptes de la plateforme, tous rôles confondus (super-admin, admin, bibliothécaire, adhérent).

- **id** – Entier – Identifiant unique – Clé primaire, auto-incrémenté
- **nom** – Texte – Nom de famille – Obligatoire
- **prenom** – Texte – Prénom – Obligatoire
- **email** – Texte – Adresse email – Obligatoire, unique
- **motdepasse** – Texte – Hash du mot de passe – Obligatoire
- **role** – Énumération – Rôle de l'utilisateur : super_admin, admin, bibliothecaire, adherent – Obligatoire
- **bibliothequeid** – Entier – Référence vers la bibliothèque – Obligatoire sauf pour superadmin (NULL autorisé)
- **date_inscription** – Date – Date de création du compte – Obligatoire, générée automatiquement
- **actif** – Booléen – Compte actif ou désactivé – Par défaut : vrai

2.3 Entité : livres

Catalogue global de tous les livres référencés sur la plateforme, indépendamment des bibliothèques. Les informations peuvent être pré-remplies via l'API Google Books.

- **id** – Entier – Identifiant unique – Clé primaire, auto-incrémenté
- **isbn** – Texte – Code ISBN-13 – Obligatoire, unique
- **titre** – Texte – Titre du livre – Obligatoire
- **auteur** – Texte – Nom de l'auteur principal – Obligatoire
- **editeur** – Texte – Nom de l'éditeur – Optionnel
- **annee_publication** – Entier – Année de publication – Optionnel
- **genre** – Texte – Genre littéraire – Optionnel
- **resume** – Texte long – Résumé du livre – Optionnel
- **couverture_url** – Texte – URL de l'image de couverture (fournie par Google Books) – Optionnel
- **date_ajout** – Date – Date d'ajout dans le catalogue – Obligatoire, générée automatiquement

2.4 Entité : exemplaires

Représente un exemplaire physique d'un livre appartenant à une bibliothèque spécifique.

- **id** – Entier – Identifiant unique – Clé primaire, auto-incrémenté
- **livre_id** – Entier – Référence vers le livre – Obligatoire, clé étrangère vers livres
- **bibliotheque_id** – Entier – Référence vers la bibliothèque propriétaire – Obligatoire, clé étrangère vers bibliotheques
- **statut** – Énumération – État de l'exemplaire : disponible, emprunte, perdu, retire – Par défaut : disponible

- **date_ajout** – Date – Date d'ajout de l'exemplaire au stock – Obligatoire, générée automatiquement
- **notes** – Texte – Observations sur l'état physique – Optionnel

2.5 Entité : prets

Représente l'emprunt d'un exemplaire par un adhérent.

- **id** – Entier – Identifiant unique – Clé primaire, auto-incrémenté
- **exemplaire_id** – Entier – Référence vers l'exemplaire emprunté – Obligatoire, clé étrangère vers exemplaires
- **adherent_id** – Entier – Référence vers l'adhérent emprunteur – Obligatoire, clé étrangère vers utilisateurs
- **bibliotheque_id** – Entier – Référence vers la bibliothèque concernée – Obligatoire, clé étrangère vers bibliotheques
- **date_emprunt** – Date – Date de début du prêt – Obligatoire, générée automatiquement
- **dateretourprevue** – Date – Date de retour attendue – Obligatoire
- **dateretoureffective** – Date – Date de retour réel – Optionnel (NULL tant que non rendu)
- **statut** – Énumération – État du prêt : encours, rendu, enretard – Par défaut : en_cours

2.6 Entité : demandes_migration

Représente une demande d'un adhérent pour changer de bibliothèque.

- **id** – Entier – Identifiant unique – Clé primaire, auto-incrémenté
- **adherent_id** – Entier – Référence vers l'adhérent demandeur – Obligatoire, clé étrangère vers utilisateurs
- **bibliothequesourceid** – Entier – Bibliothèque actuelle de l'adhérent – Obligatoire, clé étrangère vers bibliotheques
- **bibliothequecibleid** – Entier – Bibliothèque souhaitée – Obligatoire, clé étrangère vers bibliotheques
- **statut** – Énumération – État de la demande : enattente, validee, refusee – Par défaut : enattente
- **date_demande** – Date – Date de la demande – Obligatoire, générée automatiquement
- **date_traitement** – Date – Date de validation ou refus – Optionnel (NULL tant que non traitée)
- **commentaire** – Texte – Motif de refus ou remarque de l'admin – Optionnel

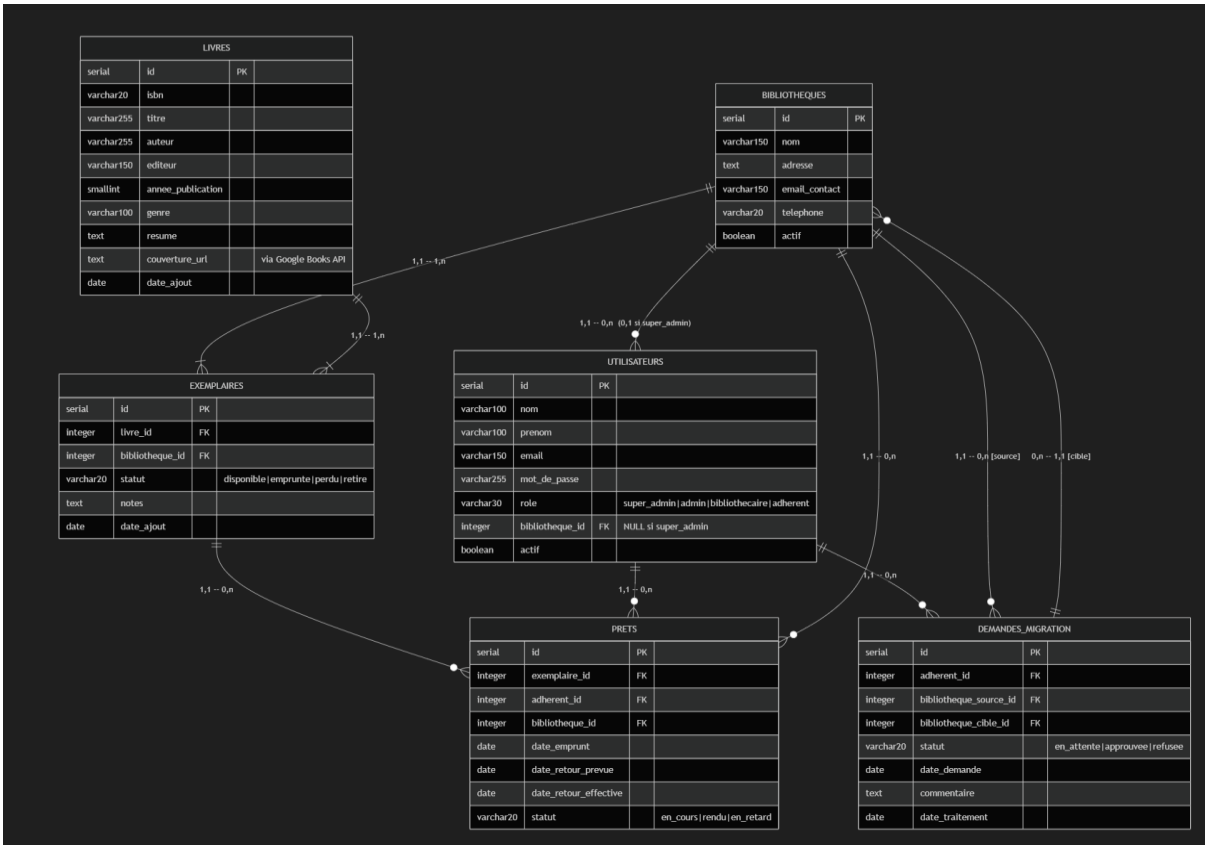
---

3) MCD – Modèle Conceptuel de Données

Rôles utilisateurs : `super_admin` (global, sans bibliothèque) · `admin` (gestion complète d'une bibliothèque) · `bibliothecaire` (livres et prêts) · `adherent` (self-service + migration)

>

`superadmin` : `bibliothequeid` est NULL — il n'appartient à aucune bibliothèque, d'où la cardinalité **(0,1)** côté utilisateur.



Relations et cardinalités

- **bibliotheques – utilisateurs** : une bibliothèque peut avoir plusieurs utilisateurs (admins, bibliothécaires, adhérents) ; un utilisateur appartient à une seule bibliothèque. *(1,1) – (0,n)*
- **bibliotheques – exemplaires** : une bibliothèque possède plusieurs exemplaires ; un exemplaire appartient à une seule bibliothèque. *(1,1) – (0,n)*
- **livres – exemplaires** : un livre peut avoir plusieurs exemplaires dans différentes bibliothèques ; un exemplaire correspond à un seul livre. *(1,1) – (0,n)*
- **exemplaires – prets** : un exemplaire peut avoir plusieurs prêts (successifs) ; un prêt porte sur un seul exemplaire. *(1,1) – (0,n)*
- **utilisateurs (adhérent) – prets** : un adhérent peut avoir plusieurs prêts ; un prêt appartient à un seul adhérent. *(1,1) – (0,n)*
- **utilisateurs (adhérent) – demandes_migration** : un adhérent peut faire plusieurs demandes (successives) ; une demande appartient à un seul adhérent. *(1,1) – (0,n)*

- **bibliotheques – demandes_migration (source)** : une bibliothèque peut recevoir plusieurs demandes de départ. *(1,1) – (0,n)*
- **bibliotheques – demandes_migration (cible)** : une bibliothèque peut recevoir plusieurs demandes d'arrivée. *(1,1) – (0,n)*

---

4) MLD – Modèle Logique de Données

Traduction du MCD en tables relationnelles. Les clés primaires sont notées **PK**, les clés étrangères **FK**.

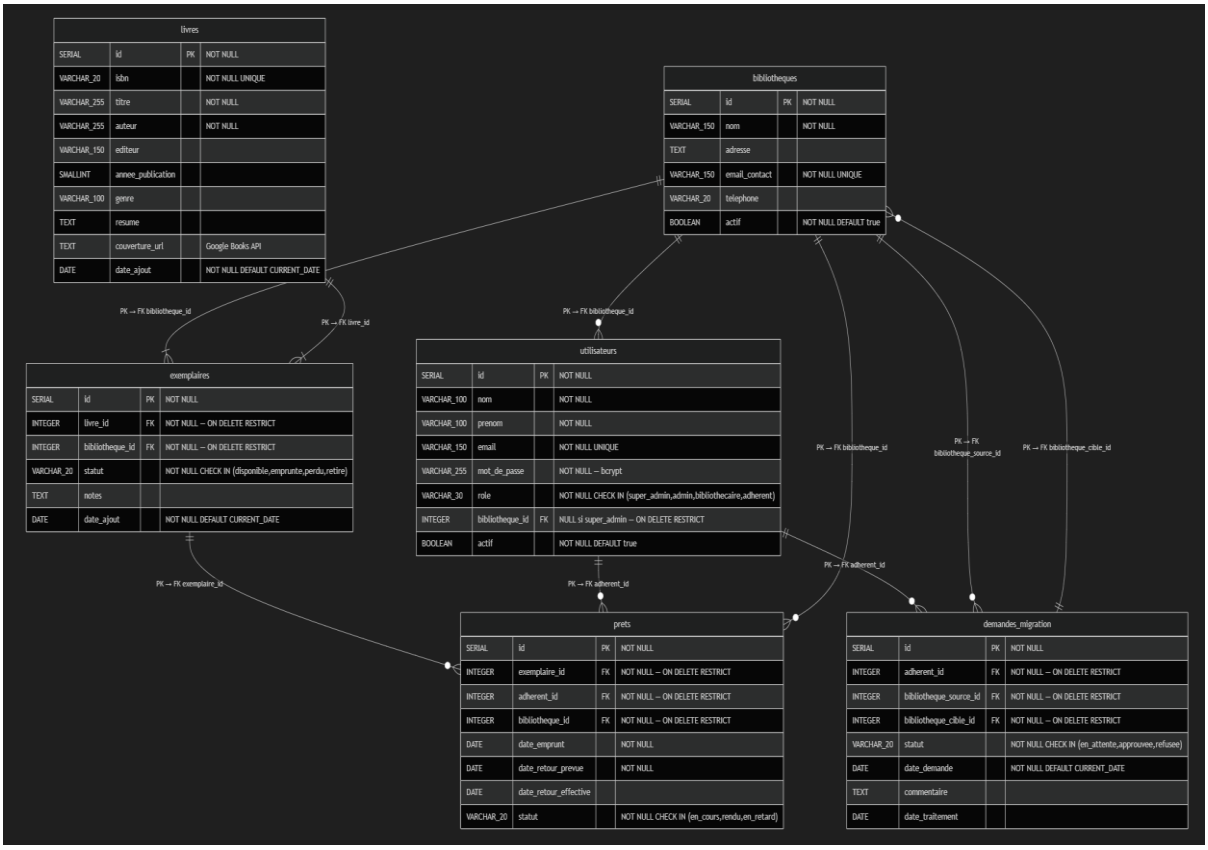
- **bibliotheques** (id PK, nom, adresse, emailcontact, telephone, dateinscription, actif)
- **utilisateurs** (id PK, nom, prenom, email, motdepasse, role, bibliothequeid FK→bibliotheques, dateinscription, actif)
- **livres** (id PK, isbn, titre, auteur, editeur, anneepublication, genre, resume, couvertureurl, date_ajout)
- **exemplaires** (id PK, livreid FK→livres, bibliothequeid FK→bibliotheques, statut, date_ajout, notes)
- **prets** (id PK, exemplaireid FK→exemplaires, adherentid FK→utilisateurs, bibliothequeid FK→bibliotheques, dateemprunt, dateretourprevue, dateretoureffective, statut)
- **demandesmigration** (id PK, adherentid FK→utilisateurs, bibliothequesourceid FK→bibliotheques, bibliothequecibleid FK→bibliotheques, statut, datedemande, datetraitement, commentaire)

---

5) MPD – Modèle Physique de Données (PostgreSQL)

MPD PostgreSQL — types physiques, contraintes et clés étrangères explicites.

Toutes les FK ont `ON DELETE RESTRICT` sauf `bibliothequeid` dans `utilisateurs` (`ON DELETE SET NULL`, pour permettre la suppression d'une bibliothèque sans supprimer le compte `superadmin`).



Script SQL – CREATE TABLE

```
CREATE TABLE bibliotheques (
    id SERIAL PRIMARY KEY,
    nom VARCHAR(255) NOT NULL,
    adresse TEXT NOT NULL,
    email_contact VARCHAR(255) NOT NULL UNIQUE,
    telephone VARCHAR(20),
    dateinscription DATE NOT NULL DEFAULT CURRENTDATE,
    actif BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE utilisateurs (
    id SERIAL PRIMARY KEY,
    nom VARCHAR(100) NOT NULL,
    prenom VARCHAR(100) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    motdepasse VARCHAR(255) NOT NULL,
    role VARCHAR(20) NOT NULL CHECK (role IN ('super_admin', 'admin', 'bibliothecaire', 'adherent')),
```

```
    bibliotheque_id INTEGER REFERENCES bibliotheques(id) ON DELETE SET NULL,
    dateinscription DATE NOT NULL DEFAULT CURRENTDATE,
    actif BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE livres (
    id SERIAL PRIMARY KEY,
    isbn VARCHAR(13) NOT NULL UNIQUE,
    titre VARCHAR(500) NOT NULL,
    auteur VARCHAR(255) NOT NULL,
    editeur VARCHAR(255),
    annee_publication SMALLINT,
    genre VARCHAR(100),
    resume TEXT,
    couverture_url TEXT,
    dateajout DATE NOT NULL DEFAULT CURRENTDATE
);

CREATE TABLE exemplaires (
    id SERIAL PRIMARY KEY,
    livre_id INTEGER NOT NULL REFERENCES livres(id) ON DELETE RESTRICT,
    bibliotheque_id INTEGER NOT NULL REFERENCES bibliotheques(id) ON DELETE RESTRICT,
    statut VARCHAR(20) NOT NULL DEFAULT 'disponible' CHECK (statut IN ('disponible', 'emprunte', 'perdu',
'retire')),
    dateajout DATE NOT NULL DEFAULT CURRENTDATE,
    notes TEXT
);

CREATE TABLE prets (
    id SERIAL PRIMARY KEY,
    exemplaire_id INTEGER NOT NULL REFERENCES exemplaires(id) ON DELETE RESTRICT,
    adherent_id INTEGER NOT NULL REFERENCES utilisateurs(id) ON DELETE RESTRICT,
    bibliotheque_id INTEGER NOT NULL REFERENCES bibliotheques(id) ON DELETE RESTRICT,
    dateemprunt DATE NOT NULL DEFAULT CURRENTDATE,
    dateretourprevue DATE NOT NULL,
    dateretoureffective DATE,
    statut VARCHAR(20) NOT NULL DEFAULT 'encours' CHECK (statut IN ('encours', 'rendu', 'en_retard'))
);

CREATE TABLE demandes_migration (
```

```
    id SERIAL PRIMARY KEY,
    adherent_id INTEGER NOT NULL REFERENCES utilisateurs(id) ON DELETE RESTRICT,
    bibliothequesourceid INTEGER NOT NULL REFERENCES bibliotheques(id) ON DELETE RESTRICT,
    bibliothequecibleid INTEGER NOT NULL REFERENCES bibliotheques(id) ON DELETE RESTRICT,
    statut VARCHAR(20) NOT NULL DEFAULT 'enattente' CHECK (statut IN ('enattente', 'validee', 'refusee')),
    datedemande DATE NOT NULL DEFAULT CURRENTDATE,
    date_traitement DATE,
    commentaire TEXT
);
```

---

6) Justifications de modélisation

- **Catalogue global + exemplaires séparés** : évite la duplication des métadonnées d'un livre présent dans plusieurs bibliothèques. Les informations (titre, auteur, ISBN) sont stockées une seule fois dans `livres`, tandis que le stock physique est géré dans `exemplaires` par bibliothèque.
- **`bibliothequeid` dans `prets`** : redondant par rapport à `exemplaire.bibliothequeid`, mais conservé pour simplifier et optimiser les requêtes de reporting par bibliothèque sans jointure supplémentaire.
- **`role` dans `utilisateurs`** : une seule table d'authentification pour tous les profils. Le filtre par `bibliotheque_id` garantit l'isolation des données entre bibliothèques (principe de moindre privilège, recommandation ANSSI).
- **`ON DELETE RESTRICT`** sur les clés étrangères critiques : empêche la suppression accidentelle d'une bibliothèque ou d'un exemplaire tant qu'il existe des prêts ou des adhérents rattachés.
- **Normalisation 3NF** : aucune dépendance transitive. Chaque attribut dépend uniquement de la clé primaire de sa table.

Jalon 4 – Conception de l'Application & Architecture

Projet : BiblioTech
Auteur : Grégory Sergent
Formation : CDA – Concepteur Développeur d'Applications
Période : Janvier → Juin 2026
Date : 30/04/2026

---

1) Introduction

Ce dossier constitue le livrable du **Jalon 4** : la conception technique de l'application BiblioTech. Il fait suite aux jalons précédents :

- **Jalon 1** : Cahier des charges fonctionnel (périmètre, acteurs, fonctionnalités)
- **Jalon 2** : Méthodologie de projet & conception UI/UX (maquettes, charte graphique)
- **Jalon 3** : Modélisation de la base de données (MCD, MLD, MPD PostgreSQL)

L'objectif de ce jalon est de traduire les besoins fonctionnels en **diagrammes UML** et en une **description d'architecture** technique, qui serviront de guide pour le développement (Jalon 5).

---

2) Diagrammes de cas d'utilisation (Use Cases UML)

2.1 Acteurs identifiés

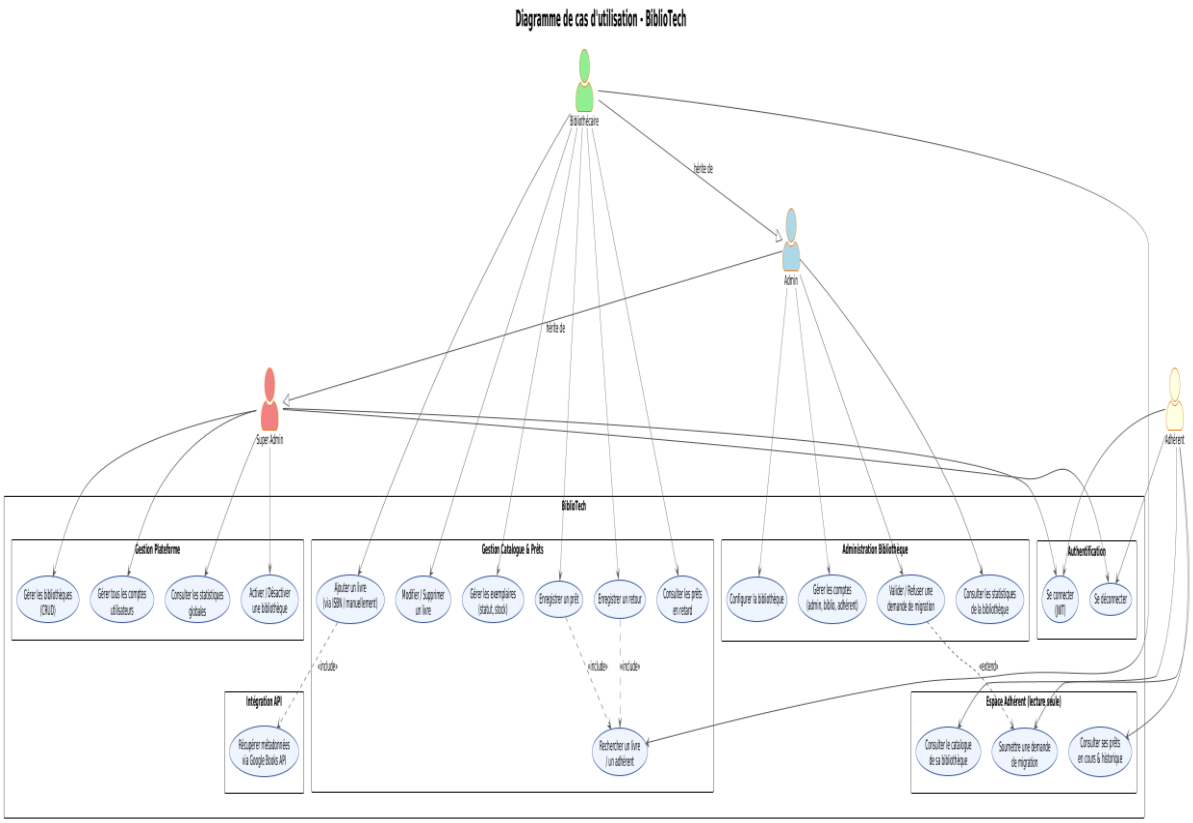
BiblioTech distingue **4 rôles** avec des périmètres d'action distincts :

- **Super Admin** : gestionnaire de la plateforme SaaS globale. N'appartient à aucune bibliothèque (`bibliotheque_id = NULL`). Gère les bibliothèques clientes et peut intervenir sur tous les comptes.
- **Admin** : gestionnaire d'une bibliothèque. Gère les comptes de sa bibliothèque, valide les demandes de migration, consulte les statistiques locales.
- **Bibliothécaire** : opérateur quotidien. Gère le catalogue (livres, exemplaires) et les prêts/retours de sa bibliothèque.

- **Adhérent** : utilisateur final en lecture seule. Consulte le catalogue et ses prêts, soumet des demandes de migration.

> Les rôles sont **cumulatifs par héritage** : le Super Admin peut tout faire, l'Admin peut tout faire sauf la gestion plateforme globale, etc.

2.2 Diagramme



2.3 Cas d'utilisation principaux

Super Admin	Gérer les bibliothèques (CRUD), gérer tous les comptes, activer/désactiver une bibliothèque, consulter stats globales
Admin	Gérer les comptes (admin/biblio/adhérent), valider/refuser une migration, configurer la bibliothèque
Bibliothécaire	Ajouter un livre (ISBN/manuel), gérer les exemplaires, enregistrer un prêt/retour, consulter les retards

Adhérent	Consulter le catalogue, consulter ses prêts, soumettre une demande de migration
Tous	Se connecter (JWT), se déconnecter

Relations `<<include>>` / `<<extend>>` :

- *Ajouter un livre* `<<include>>` *Récupérer métadonnées via Google Books API* (appel systématique si ISBN fourni)
- *Enregistrer un prêt* `<<include>>` *Rechercher un adhérent / exemplaire*
- *Valider une demande de migration* `<<extend>>` *Soumettre une demande de migration* (l'admin agit en réponse à une demande existante)

---

3) Diagrammes de séquence

3.1 Séquence 1 – Enregistrer un prêt

Ce scénario illustre le flux complet d'enregistrement d'un prêt par un bibliothécaire, depuis la saisie dans l'interface React jusqu'à la mise à jour du statut de l'exemplaire en base.

Composants impliqués : React → `PretController` → `PretService` → `ExemplaireRepository` + `PretRepository` → PostgreSQL

Points clés :

- Vérification que l'exemplaire existe et est `disponible` avant création
- Transaction : création du prêt ET mise à jour du statut exemplaire (`emprunte`)
- Gestion des cas d'erreur : exemplaire non trouvé (404), exemplaire indisponible (409)

3.2 Séquence 2 – Ajouter un livre via ISBN (Google Books API)

Ce scénario couvre l'intégration de l'API externe Google Books, qui est l'API tierce imposée par le CDC technique.

Composants impliqués : React → `LivreController` → `GoogleBooksService` → API Google Books → `LivreService` → `LivreRepository` → PostgreSQL

Points clés :

- Vérification préalable que le livre n'est pas déjà dans le catalogue (unicité ISBN)
- Appel HTTP sortant vers `https://www.googleapis.com/books/v1/volumes?q=isbn:{isbn}`
- La clé API est stockée dans la variable d'environnement `GOOGLEBOOKSAPI_KEY` (jamais en dur dans le code)
- Pré-remplissage du formulaire avec les métadonnées récupérées
- Fallback vers saisie manuelle si l'ISBN n'est pas trouvé

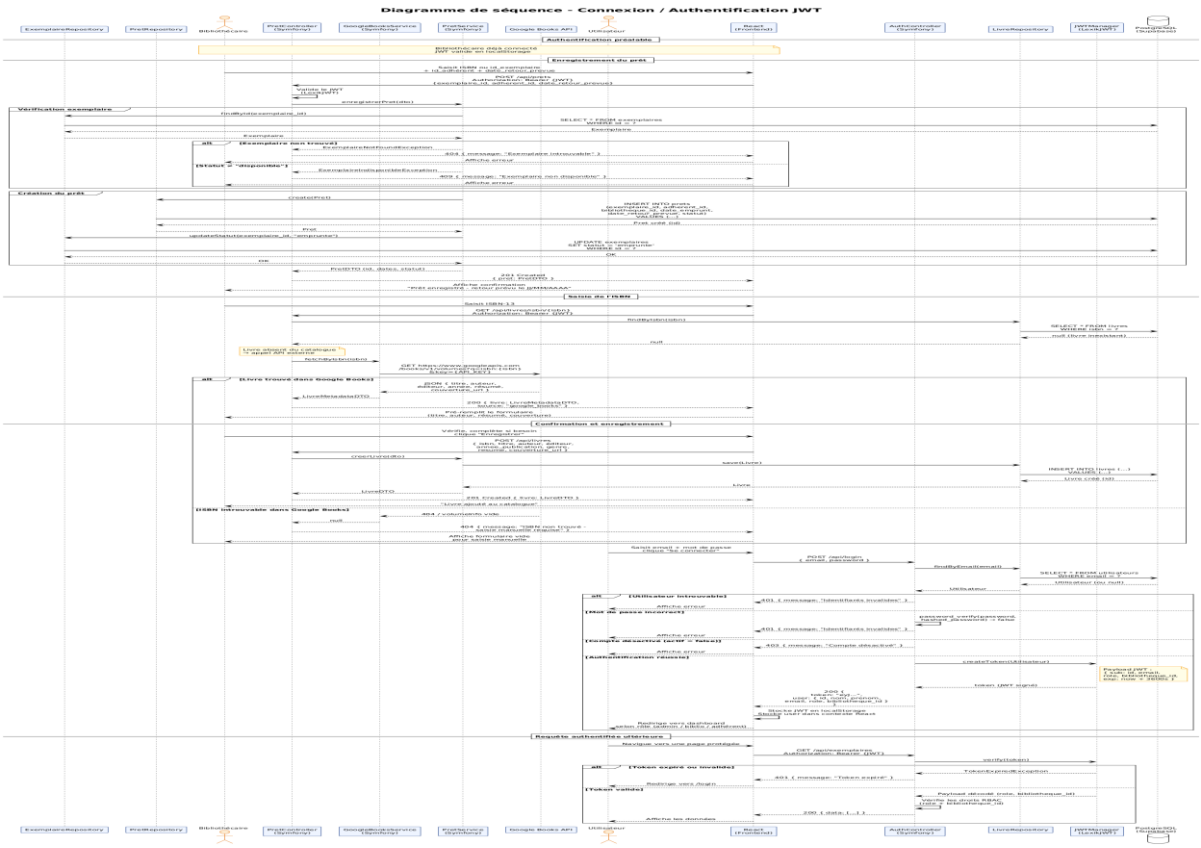
3.3 Séquence 3 – Connexion et authentification JWT

Ce scénario détaille le mécanisme d'authentification JWT (LexikJWTAuthenticationBundle) et son utilisation pour les requêtes authentifiées suivantes.

Composants impliqués : React → `AuthController` → `UtilisateurRepository` → `JWTManager` (LexikJWT)

Points clés :

- Vérification email + bcrypt/Argon2 hash du mot de passe
- Gestion des cas : identifiants invalides (401), compte désactivé (403)
- Payload JWT contient : `sub` (id), `email`, `role`, `bibliotheque_id`, `exp`
- Le frontend stocke le token en `localStorage` et l'injecte dans chaque requête via `Authorization: Bearer`
- Vérification RBAC (rôle + `bibliotheque_id`) sur chaque endpoint protégé



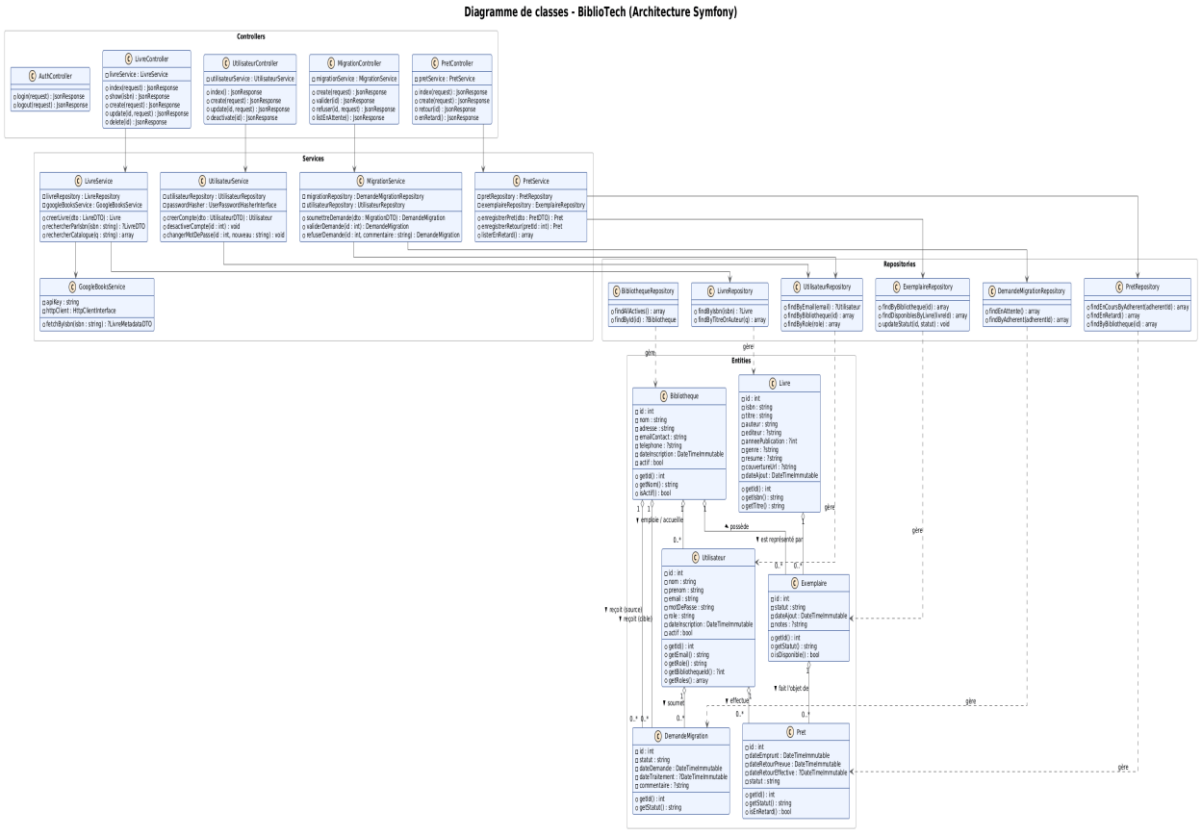
---

4) Diagramme de classes

4.1 Vue d'ensemble

Le diagramme de classes représente l'architecture interne du backend Symfony, organisée en **4 packages** :

- **Entities** : classes métier mappées par Doctrine ORM (= tables PostgreSQL du Jalon 3)
- **Repositories** : accès aux données (requêtes Doctrine DQL/SQL)
- **Services** : logique métier (règles, orchestration)
- **Controllers** : points d'entrée HTTP (routes API REST)



4.2 Correspondance Entities ↔ MPD (Jalon 3)

`Bibliotheque`	`bibliotheques`
`Utilisateur`	`utilisateurs`
`Livre`	`livres`
`Exemplaire`	`exemplaires`
`Pret`	`prets`
`DemandeMigration`	`demandes_migration`

4.3 Principes appliqués

- **Single Responsibility** : chaque classe a une responsabilité unique. Ex : `GoogleBooksService` gère uniquement les appels à l'API externe, `PretService` uniquement la logique métier des prêts.
- **Dependency Injection** : les services sont injectés dans les contrôleurs via le container Symfony (pas de `new` direct).

- **Repository Pattern** : l'accès aux données est isolé dans les repositories, les services ne font jamais de requêtes SQL directes.

---

5) Architecture multi-couches

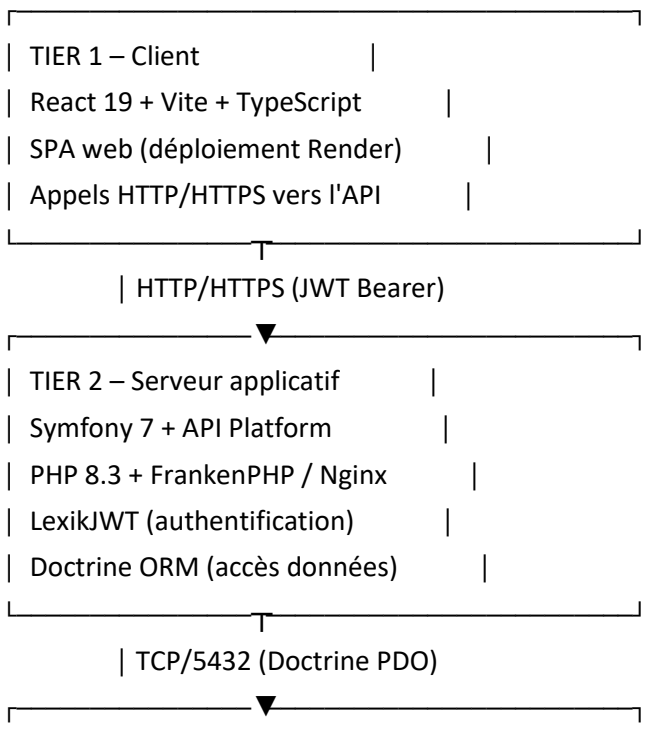
5.1 Pattern MVC avec Symfony

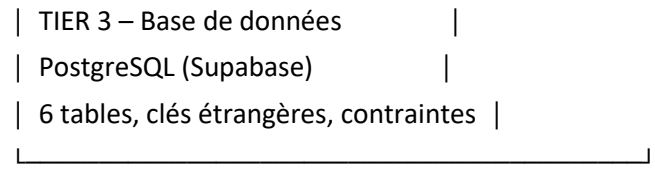
BiblioTech suit le pattern **MVC** adapté à une architecture API REST + SPA :

- **Modèle** : entités Doctrine (`src/Entity/`) + repositories (`src/Repository/`) + services (`src/Service/`)
- **Vue** : application React (frontend Tauri, côté client — aucun template Twig)
- **Contrôleur** : controllers Symfony (`src/Controller/`) exposant les routes `/api/*`

La logique métier est **entièrement encapsulée dans les Services**, les Controllers se limitant à valider la requête HTTP et sérialiser la réponse JSON.

5.2 Architecture n-tiers





Note : Le Jalon 1 envisageait Tauri pour une application desktop native. Ce choix a été abandonné au profit d'une SPA web React 19 déployée sur Render (voir note d'évolution architecturale dans le Jalon 5).

5.3 Conteneurisation Docker

Trois services Docker Compose :

- **`app`** : image PHP 8.3 + Symfony + Nginx, expose le port 8080
- **`db`** : image PostgreSQL 16 (pour le développement local — en production : Supabase cloud)
- **`frontend`** : build Tauri/React pour le développement (hot-reload)

Les variables sensibles (clé API Google Books, JWT passphrase, DSN base de données) sont stockées dans `.env.local` (jamais versionné sur Git).

5.4 API externe – Google Books

Conformément aux exigences du CDC technique, BiblioTech intègre une API tierce externe : **Google Books API v1**.

- **Usage** : récupération automatique des métadonnées (titre, auteur, éditeur, résumé, couverture) lors de l'ajout d'un livre par ISBN
- **Sécurité** : clé API dans variable d'environnement `GOOGLEBOOKSAPI_KEY`
- **Composant** : classe `GoogleBooksService` (voir diagramme de classes §4)
- **Fallback** : si l'ISBN n'est pas trouvé, formulaire de saisie manuelle proposé

5.5 Sécurité – Vue d'ensemble

Injection SQL	Doctrine ORM + requêtes préparées (aucun SQL dynamique sans paramètres)
XSS	React échappe les variables par défaut (pas de `dangerouslySetInnerHTML`)
CSRF	API REST stateless + JWT Bearer (pas de cookie session)

Brute force	Rate Limiter Symfony sur `/api/login`
Mots de passe	Hachage Argon2id via `UserPasswordHasher` Symfony
Secrets	Variables d'environnement `.env.local`, jamais en dur dans le code
RBAC	Vérification `role` + `bibliotheque_id` sur chaque endpoint Symfony

---

6) Endpoints de l'API REST

L'API BiblioTech expose des routes REST sous le préfixe `/api`. Toutes les routes (sauf `/api/login`) nécessitent un header `Authorization: Bearer <token>`. Le contrôle d'accès est basé sur le champ `role` du payload JWT et, si applicable, sur `bibliotheque_id`.

6.1 Authentification

POST	`/api/login`	Aucun (public)	Authentification — retourne un token JWT
POST	`/api/logout`	Tout rôle	Invalidation côté client (suppression du token localStorage)

Body POST `/api/login` :

```
{ "email": "user@example.com", "password": "motdepasse" }
```

Réponse 200 :

```
{ "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiJ9..." }
```

6.2 Bibliothèques

GET	`/api/bibliotheques`	`super_admin`	Liste toutes les bibliothèques de la plateforme
POST	`/api/bibliotheques`	`super_admin`	Crée une nouvelle bibliothèque cliente
GET	`/api/bibliotheques/{id}`	`super_admin`, `admin` (propre)	Détail d'une bibliothèque
PUT	`/api/bibliotheques/{id}`	`super_admin`, `admin` (propre)	Modifie les informations d'une bibliothèque
DELETE	`/api/bibliotheques/{id}`	`super_admin`	Supprime (désactive) une bibliothèque
PATCH	`/api/bibliotheques/{id}/activer`	`super_admin`	Active ou désactive une bibliothèque

6.3 Utilisateurs

GET	`/api/utilisateurs`	`super_admin`, `admin`	Liste les utilisateurs (filtrés par `bibliotheque_id` pour admin)
POST	`/api/utilisateurs`	`super_admin`, `admin`	Crée un compte utilisateur
GET	`/api/utilisateurs/{id}`	`super_admin`, `admin`, soi-même	Détail d'un utilisateur
PUT	`/api/utilisateurs/{id}`	`super_admin`, `admin`, soi-même	Modifie un compte
DELETE	`/api/utilisateurs/{id}`	`super_admin`, `admin`	Supprime un compte
GET	`/api/utilisateurs/me`	Tout rôle	Profil de l'utilisateur connecté (issu du JWT)

6.4 Livres

GET	`/api/livres`	Tout rôle	Liste le catalogue de la bibliothèque (filtres : titre, auteur, ISBN)
POST	`/api/livres`	`bibliothecaire`, `admin`	Ajoute un livre au catalogue
GET	`/api/livres/{id}`	Tout rôle	Détail d'un livre
PUT	`/api/livres/{id}`	`bibliothecaire`, `admin`	Modifie les informations d'un livre
DELETE	`/api/livres/{id}`	`admin`	Supprime un livre du catalogue
GET	`/api/livres/isbn/{isbn}`	`bibliothecaire`, `admin`	Recherche via Google Books API et retourne les métadonnées

---

6.5 Exemplaires

GET	`/api/exemplaires`	Tout rôle	Liste les exemplaires (filtres : `livre_id`, `statut`)
POST	`/api/exemplaires`	`bibliothecaire`, `admin`	Crée un exemplaire pour un livre existant
GET	`/api/exemplaires/{id}`	Tout rôle	Détail d'un exemplaire
PUT	`/api/exemplaires/{id}`	`bibliothecaire`, `admin`	Modifie un exemplaire (statut, état)
DELETE	`/api/exemplaires/{id}`	`admin`	Supprime un exemplaire

6.6 Prêts

GET	`/api/prets`	`bibliothecaire`, `admin`	Liste les prêts (filtres : `adherent_id`, `statut`, `retard`)
POST	`/api/prets`	`bibliothecaire`	Enregistre un nouveau prêt
GET	`/api/prets/{id}`	`bibliothecaire`, `admin`, adhérent concerné	Détail d'un prêt
PATCH	`/api/prets/{id}/retour`	`bibliothecaire`	Enregistre le retour d'un exemplaire
GET	`/api/prets/retards`	`bibliothecaire`, `admin`	Liste les prêts en retard
GET	`/api/prets/adherent/{id}`	`bibliothecaire`, `admin`, adhérent concerné	Historique des prêts d'un adhérent

---

6.7 Demandes de migration

GET	`/api/migrations`	`admin`, `super_admin`	Liste les demandes de migration
POST	`/api/migrations`	`adherent`	Soumet une demande de migration vers une autre bibliothèque
GET	`/api/migrations/{id}`	`admin`, `super_admin`, adhérent concerné	Détail d'une demande
PATCH	`/api/migrations/{id}/valider`	`admin`	Valide la migration (change la `bibliotheque_id` de l'adhérent)
PATCH	`/api/migrations/{id}/refuser`	`admin`	Refuse la demande de migration

6.8 Statistiques

GET	`/api/stats/bibliotheque`	`admin`	Statistiques de la bibliothèque (nb livres, prêts en cours, retards, adhérents)
GET	`/api/stats/globales`	`super_admin`	Statistiques globales de la plateforme (nb bibliothèques, prêts totaux, utilisateurs)

---

7) Stratégie de tests

7.1 Objectifs

La stratégie de tests vise à garantir la **fiabilité** et la **non-régression** de l'application à trois niveaux : unitaire, intégration et end-to-end. Elle s'appuie sur les outils standards de l'écosystème Symfony et React.

7.2 Types de tests et outils

Tests unitaires backend	PHPUnit 11	Services (PretService, GoogleBooksService, etc.)	Unitaire
Tests d'intégration API	Symfony WebTestCase / ApiTestCase	Endpoints REST (Controllers + BDD de test)	Intégration
Tests unitaires frontend	Vitest + React Testing Library	Composants React (rendering, props, events)	Unitaire
Tests E2E	Playwright	Scénarios utilisateur complets (login → prêt → retour)	End-to-End

7.3 Couverture cible par composant

`PretService`	Unitaire (PHPUnit)	Prêt OK, exemplaire indisponible, adhérent invalide

`GoogleBooksService`	Unitaire (PHPUnit + mock HTTP)	ISBN trouvé, ISBN inconnu, timeout API
`AuthController`	Intégration (WebTestCase)	Login OK (200 + JWT), mauvais mdp (401), compte désactivé (403)
`LivreController`	Intégration (WebTestCase)	CRUD complet, accès interdit selon rôle (403)
`PretController`	Intégration (WebTestCase)	POST prêt OK, POST doublon (409), PATCH retour
`UtilisateurController`	Intégration (WebTestCase)	Création, modification, suppression, accès `/me`
Composant `LoginForm`	Unitaire (Vitest)	Rendu correct, soumission formulaire, affichage erreur
Composant `CatalogueLivres`	Unitaire (Vitest)	Affichage liste, filtres, recherche ISBN
Scénario Prêt complet	E2E (Playwright)	Login biblio → recherche adhérent → enregistrer prêt → retour
Scénario Login	E2E (Playwright)	Login OK → redirection dashboard, Login KO → message erreur

7.4 Organisation des tests

Backend (Symfony) :

```
dev/backend/
├── tests/
│   ├── Unit/
│   │   ├── Service/
│   │   │   ├── PretServiceTest.php
│   │   │   └── GoogleBooksServiceTest.php
│   │   └── Entity/
│   │       └── PretTest.php
│   └── Integration/
│       ├── Controller/
│       │   ├── AuthControllerTest.php
│       │   ├── LivreControllerTest.php
│       │   └── PretControllerTest.php
│       └── fixtures/          ← données de test (DataFixtures Doctrine)
```

Frontend (React) :

```
dev/frontend/
├── src/
│   └── __tests__/
│       ├── components/
│       │   ├── LoginForm.test.tsx
│       │   └── CatalogueLivres.test.tsx
│       └── hooks/
│           └── useAuth.test.ts
└── e2e/                       ← tests Playwright
    ├── login.spec.ts
```

```
    └── pret.spec.ts
```

7.5 Base de données de test

Les tests d'intégration utilisent une base PostgreSQL **dédiée** (ou SQLite en mémoire pour rapidité), alimentée par des **DataFixtures Doctrine** :

- Fixtures : 1 bibliothèque, 1 admin, 1 bibliothécaire, 2 adhérents, 5 livres, 10 exemplaires
- Réinitialisation de la base avant chaque suite de tests (`bin/console doctrine:database:drop --force --env=test`)

7.6 Intégration continue (CI)

Les tests seront exécutés automatiquement à chaque push via **GitHub Actions** :

- Job `test-backend` : `composer install` → `php bin/phpunit`
- Job `test-frontend` : `npm install` → `npm run test`
- Job `e2e` : `playwright install` → `playwright test` (sur environnement Docker)

---

8) Gestion des erreurs API

8.1 Format standard des réponses d'erreur

Toutes les erreurs retournées par l'API suivent un format JSON uniforme, facilitant leur traitement côté frontend React :

```
{
  "status": 404,
  "code": "LIVRE_NOT_FOUND",
  "message": "Le livre avec l'identifiant 42 n'existe pas.",
  "details": null
}
```

Le champ `details` peut contenir des informations supplémentaires (ex : liste des champs invalides lors d'une erreur de validation).

8.2 Codes HTTP utilisés

200 OK	Requête réussie	GET, PUT réussis
201 Created	Ressource créée	POST prêt, POST livre
204 No Content	Suppression réussie	DELETE utilisateur
400 Bad Request	Données invalides (validation)	Email mal formé, champ manquant
401 Unauthorized	Token absent ou invalide	Requête sans JWT, token expiré
403 Forbidden	Accès interdit (RBAC)	Admin tente d'accéder aux stats globales
404 Not Found	Ressource inexistante	Livre, exemplaire ou prêt introuvable
409 Conflict	Conflit métier	Exemplaire déjà emprunté, ISBN déjà dans le catalogue
422 Unprocessable Entity	Règle métier violée	Prêt impossible (adhérent a déjà 3 prêts en cours)
500 Internal Server Error	Erreur serveur inattendue	Echec appel Google Books API, erreur BDD

8.3 Catalogue des erreurs métier

AUTHINVALIDCREDENTIALS	401	Email ou mot de passe incorrect
AUTHACCOUNTDISABLED	403	Compte désactivé par un admin
AUTHTOKENEXPIRED	401	Token JWT expiré
LIVRENOTFOUND	404	Livre introuvable en base
LIVREISBNDUPLICATE	409	Un livre avec cet ISBN existe déjà
LIVREISBNNOTFOUNDGOOGLE	404	ISBN non trouvé dans Google Books API
EXEMPLAIRENOTFOUND	404	Exemplaire introuvable
EXEMPLAIRE_UNAVAILABLE	409	Exemplaire déjà emprunté ou hors service
PRETNOTFOUND	404	Prêt introuvable
PRETALREADYRETURNED	409	Retour déjà enregistré pour ce prêt

PRETMAXREACHED	422	L'adhérent a atteint le nombre maximum de prêts simultanés
USERNOTFOUND	404	Utilisateur introuvable
USEREMAILDUPLICATE	409	Un compte avec cet email existe déjà
MIGRATIONNOTFOUND	404	Demande de migration introuvable
MIGRATIONALREADYPROCESSED	409	Demande déjà validée ou refusée
BIBLIOTHEQUENOTFOUND	404	Bibliothèque introuvable
ACCESS_DENIED	403	Action non autorisée pour ce rôle
GOOGLEBOOKSAPI_ERROR	500	Echec de la communication avec Google Books API

---

9) État d'avancement du développement

La phase de conception étant finalisée avec ce livrable, le développement débutera immédiatement après validation du Jalon 4. La structure du projet a été mise en place pour permettre un démarrage rapide :

- Structure des dossiers du projet créée (`dev/backend`, `dev/frontend`)
- Architecture définie et documentée dans ce livrable (Controllers, Services, Entities, Repositories)
- Base de données modélisée et prête à être implémentée (MPD Jalon 3)
- Docker Compose à configurer (services : `app`, `db`, `frontend`)

Le développement complet des fonctionnalités sera livré au **Jalon 5 (mai 2026)**.

Jalon 5 – Développement, Sécurité & Tests

Projet : BiblioTech
Auteur : Grégory Sergent
Organisme de formation : IPSSI Grande École d'Informatique
Formation : CDA – Concepteur Développeur d'Applications
Période : Janvier → Juin 2026
Date : 1er juin 2026 *(rendu avec 1 jour de retard — fonctionnalités finalisées le 31 mai)*
Statut : Version 1.1

---

1) Introduction

Ce livrable constitue le **Jalon 5** du projet BiblioTech. Il couvre le développement complet du backend Symfony, l'implémentation des mesures de sécurité, et la stratégie de tests appliquée.

> **Note d'évolution architecturale — Tauri → Application web**
> Le Jalon 1 envisageait BiblioTech comme une application **desktop native Tauri** (livrable `.exe`/`.msi`, mode hors connexion partiel). Au cours du développement, ce choix a été abandonné au profit d'une **SPA React + Vite déployée sur le web** (Render), pour trois raisons : (1) la complexité du pipeline de build Tauri sur Windows en développement solo dépassait le périmètre pédagogique du projet ; (2) un déploiement web simplifie considérablement la CI/CD, la mise en production et la démonstration aux évaluateurs ; (3) les fonctionnalités clés — catalogue, gestion des prêts, administration — ne nécessitent pas d'accès hors ligne. Les contraintes initiales d'installateur natif et de mode offline ont donc été abandonnées. L'application est conforme au CDC technique pour toutes les autres exigences (API REST Symfony, React, PostgreSQL, Docker, CI/CD, sécurité, tests).

Les jalons précédents avaient posé les bases :

- **Jalon 1** : Cahier des charges fonctionnel
- **Jalon 2** : Méthodologie & conception UI/UX (maquettes, charte graphique)
- **Jalon 3** : Modélisation de la base de données (MCD, MLD, MPD PostgreSQL)
- **Jalon 4** : Conception UML & architecture technique (use cases, séquences, classes, endpoints REST)

Le **Jalon 5** marque la réalisation concrète :

- Implémentation de l'**API REST Symfony 7** (Controllers, Services, Entities, Repositories)
- Implémentation des mécanismes de **sécurité** (JWT, RBAC, OWASP, brute-force protection)

- Développement de fonctionnalités avancées : **premier login**, **CGU versionnées avec CMS**, **prêts perdus**, **durée de prêt configurable**
- Rédaction et exécution des **tests unitaires et d'intégration**
- Le **frontend React** est désormais **entièrement connecté au backend** via axios (appels réels à l'API REST)

---

2) Ce qui a été développé

2.1 Vue d'ensemble

L'API BiblioTech suit une architecture **MVC stricte** sur Symfony 7 + API Platform :

```
backend/
├── src/
│   ├── Controller/         # Points d'entrée HTTP (/api/*)
│   │   ├── AuthController.php
│   │   ├── BibliothequeController.php
│   │   ├── CguController.php           ← nouveau (versionnement CGU)
│   │   ├── UtilisateurController.php
│   │   ├── LivreController.php
│   │   ├── ExemplaireController.php
│   │   ├── PretController.php
│   │   ├── DemandeMigrationController.php
│   │   └── StatController.php
│   ├── Entity/             # Entités Doctrine (= tables PostgreSQL)
│   │   ├── Bibliotheque.php            ← duretPretJours ajouté
│   │   ├── Utilisateur.php             ← mustChangePassword, cguAcceptedVersion,
loginAttempts, lockedUntil, pretsSuspendus
│   │   ├── Livre.php
│   │   ├── Exemplaire.php              ← STATUT_PERDU ajouté
│   │   ├── Pret.php                    ← STATUT_PERDU, etatDepart, etatRetour ajoutés
│   │   ├── CguVersion.php              ← nouveau (versionnement CGU)
│   │   ├── DemandeMigration.php
│   │   └── RefreshToken.php
│   ├── Repository/         # Requêtes Doctrine
│   │   ├── BibliothequeRepository.php
│   │   ├── UtilisateurRepository.php
│   │   ├── LivreRepository.php
│   │   ├── ExemplaireRepository.php
│   │   ├── PretRepository.php
│   │   ├── CguVersionRepository.php    ← nouveau
│   │   ├── DemandeMigrationRepository.php
│   │   └── RefreshTokenRepository.php
│   ├── Service/            # Logique métier
│   │   ├── PretService.php             ← mise à jour (prêts perdus, durée configurable)
│   │   └── GoogleBooksService.php
│   └── EventSubscriber/    # Écouteurs d'événements Symfony
│       └── JwtCreatedSubscriber.php    ← payload étendu
├── config/
│   └── packages/
│       ├── security.yaml
│       ├── framework.yaml              ← rate limiter configuré
│       ├── doctrine.yaml
```

```
│       └── lexik_jwt_authentication.yaml
├── migrations/
│   ├── Version20260501000000.php
│   ├── Version20260526000000.php       ← refresh tokens
│   ├── Version20260601000000.php       ← mustChangePassword, loginAttempts, lockedUntil
│   ├── Version20260602000000.php       ← cguAcceptedVersion, pretsSuspendus,
duretPretJours
│   └── Version20260603000000.php       ← table cgu versions, etatDepart/etatRetour sur
prêts
└── tests/
    ├── Unit/
    │   └── Service/
    │       ├── PretServiceTest.php
    │       └── GoogleBooksServiceTest.php
    └── Integration/
        └── Controller/
            ├── AuthControllerTest.php
            ├── BibliothequeControllerTest.php
            ├── ExemplaireControllerTest.php
            ├── LivreControllerTest.php
            ├── PretControllerTest.php
            └── UtilisateurControllerTest.php
```

2.2 Entités Doctrine développées

`Bibliotheque`	`bibliotheques`	`duretpretjours` (durée de prêt configurable par bibliothèque)
`Utilisateur`	`utilisateurs`	`mustchangepassword`, `cguacceptedversion`, `pretssuspendus`, `loginattempts`, `locked_until`
`Livre`	`livres`	—
`Exemplaire`	`exemplaires`	constantes `STATUTINDISPONIBLE`, `STATUTPERDU`
`Pret`	`prets`	`STATUTPERDU`, `etatdepart`, `etat_retour`
`CguVersion`	`cgu_versions`	`version`, `contenu` (TEXT brut, paragraphes séparés par `\n`), `dateeffet`, `datepublication`, `publie_par`
`DemandeMigration`	`demandes_migration`	—
`RefreshToken`	`refresh_tokens`	—

2.3 Endpoints implémentés

`AuthController`

- `POST /api/login` — Authentification, retour JWT + refresh token. Incrémente `login_attempts`. Si ≥ 5 tentatives → verrou 15 minutes (HTTP 429).
- `POST /api/token/refresh` — Rotation du refresh token

- `POST /api/logout` — Invalidation du refresh token

`BibliothequeController` *(super_admin)*

- `GET /api/bibliotheques` — Liste toutes les bibliothèques
- `POST /api/bibliotheques` — Création
- `GET /api/bibliotheques/{id}` — Détail
- `PUT /api/bibliotheques/{id}` — Modification (dont `duretpretjours`)
- `DELETE /api/bibliotheques/{id}` — Suppression
- `PATCH /api/bibliotheques/{id}/activer` — Activation/désactivation

`CguController` *(nouveau)*

- `GET /api/cgu/current` — Version CGU actuellement en vigueur
- `GET /api/cgu/pending` — CGU en préavis (publiée mais pas encore en vigueur)
- `GET /api/cgu/history` — Historique de toutes les versions
- `POST /api/cgu` *(superadmin)* — Publication d'une nouvelle version (validation backend : dateeffet >= aujourd'hui + 15 jours, sinon HTTP 422)

`UtilisateurController`

- `GET /api/utilisateurs` — Liste (filtrée par `bibliotheque_id` pour admin/bibliothécaire)
- `POST /api/utilisateurs` — Création de compte
- `GET /api/utilisateurs/{id}` — Détail
- `PUT /api/utilisateurs/{id}` — Modification
- `DELETE /api/utilisateurs/{id}` — Suppression
- `GET /api/utilisateurs/me` — Profil de l'utilisateur connecté
- `POST /api/utilisateurs/me/accept-cgu` — Enregistrement de l'acceptation des CGU (`cguacceptedversion`)
- `POST /api/utilisateurs/{id}/changer-mot-de-passe` — Changement de mot de passe (premier login)
- `PATCH /api/utilisateurs/{id}/suspendre` — Suspendre/réactiver les prêts d'un adhérent

`LivreController`

- `GET /api/livres` — Catalogue (accessible à tous)
- `POST /api/livres` — Ajout manuel ou via ISBN
- `GET /api/livres/{id}` — Détail
- `PUT /api/livres/{id}` — Modification
- `DELETE /api/livres/{id}` — Suppression
- `GET /api/livres/isbn/{isbn}` — Recherche Google Books API

`ExemplaireController`

- `GET /api/exemplaires` — Liste avec filtres
- `POST /api/exemplaires` — Création
- `GET /api/exemplaires/{id}` — Détail

- `PUT /api/exemplaires/{id}` — Modification
- `DELETE /api/exemplaires/{id}` — Suppression
- `PATCH /api/exemplaires/{id}/perdu` *(nouveau)* — Marquer un exemplaire comme perdu

`PretController`

- `GET /api/prets` — Liste des prêts (filtrés par rôle)
- `POST /api/prets` — Enregistrement d'un prêt
- `GET /api/prets/{id}` — Détail d'un prêt
- `PATCH /api/prets/{id}/retour` — Enregistrement du retour
- `GET /api/prets/retards` — Prêts en retard
- `GET /api/prets/adherent/{id}` — Historique d'un adhérent

`DemandeMigrationController`

- `GET /api/migrations` — Liste des demandes
- `POST /api/migrations` — Soumettre une demande
- `GET /api/migrations/{id}` — Détail
- `PATCH /api/migrations/{id}/valider` — Validation
- `PATCH /api/migrations/{id}/refuser` — Refus

`StatController`

- `GET /api/stats/bibliotheque` — Stats de la bibliothèque (admin)
- `GET /api/stats/globales` — Stats globales (super_admin)

2.4 Services métier

`PretService` — Logique complète de gestion des prêts :

- Vérification de la disponibilité de l'exemplaire avant création
- Transaction atomique : création du prêt + mise à jour statut exemplaire (`emprunte`)
- Vérification du nombre maximum de prêts simultanés par adhérent (règle métier : 5 max)
- Calcul de la date de retour prévue selon `duretpretjours` de la bibliothèque (J+21 par défaut, configurable par bibliothèque)
- Enregistrement du retour : mise à jour statut exemplaire → `disponible`
- Gestion du statut `perdu` : exemplaire et prêt marqués `perdu`, date de retour non renseignée
- Détection automatique des prêts en retard (`dateretourprevue < now()`)

`GoogleBooksService` — Intégration API externe :

- Appel HTTP vers `https://www.googleapis.com/books/v1/volumes?q=isbn:{isbn}`
- Clé API depuis variable d'environnement `GOOGLEBOOKSAPI_KEY` (jamais en dur)
- Mapping des champs retournés (titre, auteur, éditeur, description, couverture)
- Gestion des cas : ISBN introuvable (404), timeout (500), quota dépassé
- Fallback documenté : si l'appel échoue, l'utilisateur bascule sur la saisie manuelle

> **Note :** Le service `GoogleBooksService` est pleinement implémenté et fonctionnel — il effectue de vraies requêtes HTTP vers `googleapis.com`. La variable d'environnement `GOOGLEBOOKSAPI_KEY` doit être renseignée dans `.env.local` (développement) ou dans les secrets du service de déploiement (production). La fonctionnalité est active dès que la clé est présente. En l'absence de clé, le service lève une exception explicite (`503`) et l'interface bascule automatiquement sur la saisie manuelle.

2.5 Frontend React (état d'avancement)

Le frontend React est fonctionnel, complet et entièrement connecté au backend.

Pages existantes :

`LoginPage`	✅ Complet	Tous — message explicite en cas de blocage brute-force (429)
`PremierLoginMotDePassePage`	✅ Complet	Tous — redirection forcée au premier login
`PremierLoginCguPage`	✅ Complet	Tous — acceptation CGU obligatoire (version dynamique)
`DashboardPage`	✅ Complet	Staff + adhérent
`CataloguePage`	✅ Complet	Tous
`PretsPage`	✅ Complet	Staff — affiche "PERDU" sans date de rendu pour les prêts perdus
`AdherentsPage`	✅ Complet	Admin + super_admin — gestion des suspensions
`DemandeMigrationPage`	✅ Complet	Tous
`BibliothequeManagementPage`	✅ Complet	Admin (sa propre bibliothèque) + super_admin (toutes) — durée de prêt configurable
`MonComptePage`	✅ Complet	Tous — profil, changement mdp, zone danger + section **Gestion CGU** (super_admin uniquement : historique + formulaire publication nouvelle version)

Composants et contextes :

`AuthContext`	✅ Complet	Gestion du JWT, refresh silencieux, flags `mustchangepassword`, `cguacceptedversion`
`CguContext`	✅ Complet	Expose `currentVersion`, `cguContenu`, `pendingVersion`, `pendingDateEffet` (CGU en préavis)
`ProtectedRoute`	✅ Complet	Guards : authentification + redirection premier login + redirection CGU
`Sidebar`	✅ Complet	Navigation filtrée par rôle — onglet Bibliothèque accessible aux admins
`PageLayout`	✅ Complet	Layout responsive desktop/mobile

Services frontend :

`authService`	✅
`utilisateurService`	✅
`livreService`	✅
`exemplaireService`	✅
`pretService`	✅
`bibliothequeService`	✅
`demandesMigrationService`	✅
`cguService`	✅
`statsService`	✅

---

3) Fonctionnalités avancées implémentées

3.1 Système de premier login (mot de passe + CGU)

Lors de la création d'un compte par un admin/superadmin, le flag `mustchange_password = true` est positionné sur l'utilisateur. Au premier login :

10. Le JWT retourné contient `mustchangepassword: true`
11. Le `ProtectedRoute` intercepte toute navigation et redirige vers `/premier-login/mot-de-passe`
12. L'utilisateur définit son propre mot de passe (validation : 8 caractères min, majuscule, chiffre, caractère spécial)
13. Après changement : redirection vers `/premier-login/cgu`
14. L'utilisateur lit et accepte les CGU (version dynamique)
15. `cguacceptedversion` est mis à jour en base via `POST /api/utilisateurs/me/accept-cgu`
16. Accès normal à l'application

Ce flux garantit qu'aucun utilisateur ne peut utiliser l'application sans avoir explicitement changé son mot de passe temporaire et accepté les CGU en vigueur.

3.2 Protection anti-brute force

Le login est protégé contre les attaques par force brute à deux niveaux :

Niveau applicatif (Symfony) — `AuthController.php` :

- Chaque tentative échouée incrémente `login_attempts` sur l'entité `Utilisateur`
- À partir de 5 tentatives : `locked_until = now() + 15 minutes`
- Si le compte est verrouillé, toute tentative retourne immédiatement HTTP 429 avec le message et la date de déverrouillage

Niveau infrastructure (Symfony Rate Limiter) — `framework.yaml` :

- Rate limiter global sur `/api/login` par adresse IP
- Limite : 10 tentatives par tranche de 5 minutes

Côté frontend — `LoginPage.tsx` :

- Réponse HTTP 429 → affichage d'un message explicite : *"Trop de tentatives infructueuses. Compte temporairement verrouillé. Réessayez dans X minutes."*

3.3 Versionnement des CGU & CMS intégré

BiblioTech dispose d'un système complet de gestion des CGU sans intervention sur le code :

Fonctionnement :

- Les CGU sont stockées en base dans la table `cgu_versions` (texte brut, paragraphes séparés par `\n`)
- Le `super_admin` publie une nouvelle version via la section **Gestion CGU** de `MonComptePage` (formulaire : numéro de version, contenu, date d'effet)
- La nouvelle version entre en vigueur **15 jours après publication minimum** (préavis obligatoire, validé côté backend : HTTP 422 si délai insuffisant)
- Durant ces 15 jours, une **bannière de préavis** est affichée dans la Sidebar à tous les utilisateurs connectés (via `CguContext.pendingVersion`)
- À la date d'effet, les utilisateurs dont `cguacceptedversion ≠ version_courante` sont redirigés vers la page d'acceptation
- Après acceptation, `POST /api/utilisateurs/me/accept-cgu` met à jour `cguacceptedversion` en base

Contraintes techniques respectées :

- Contenu texte brut (paragraphes séparés par retours à la ligne) — rendu direct dans l'UI sans parsing JSON
- Historique complet des versions consultable via `GET /api/cgu/history`
- Version et date affichées dynamiquement sur la page d'acceptation (pas de valeur en dur)

3.4 Gestion des prêts perdus

Un exemplaire peut être déclaré perdu par un bibliothécaire. Le processus :

17. `PATCH /api/exemplaires/{id}/perdu` → statut exemplaire → `perdu`
18. Le prêt associé passe au statut `perdu`
19. `etat_retour` reste `null` (le livre n'a pas été rendu)
20. **Côté frontend** : la colonne "Date de rendu" affiche **"PERDU"** et non une date, ce qui serait contradictoire

3.5 Durée de prêt configurable par bibliothèque

Chaque bibliothèque peut définir sa propre durée de prêt standard :

- Champ `duretpretjours` sur l'entité `Bibliotheque` (défaut : 21 jours)
- Configurable par le `super_admin` (toutes bibliothèques) ou par l'`admin` (sa propre bibliothèque) depuis la page `BibliothequeManagementPage`
- `PretService` utilise cette valeur pour calculer `dateretourprevue` à la création du prêt
- La valeur est incluse dans le payload JWT (`duretpretjours`) pour éviter un appel API supplémentaire

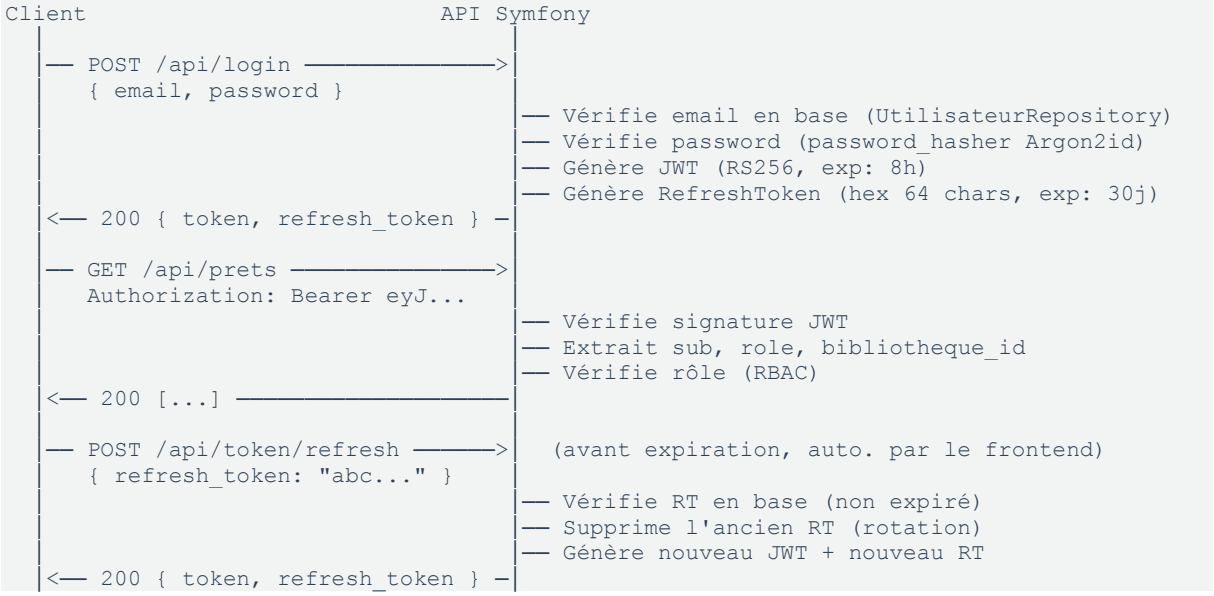
---

4) Sécurité

4.1 Authentification JWT & Refresh Token

BiblioTech utilise **LexikJWTAuthenticationBundle** pour l'authentification sans état (stateless), complété par un système de **refresh token** pour éviter les déconnexions intempestives.

Flux d'authentification :



Payload JWT étendu :

```
{
  "sub": 42,
  "email": "bibliothecaire@ipssitheque.fr",
  "role": "bibliothecaire",
  "bibliotheque_id": 1,
  "bibliotheque_nom": "IPSSIthèque",
  "must_change_password": false,
  "cgu_accepted_version": "1.1",
  "prets_suspendus": false,
  "duret_pret_jours": 21,
  "iat": 1746691200,
  "exp": 1746720000
}
```

Le payload custom (`role`, `bibliothequeid`) est injecté via un `JwtCreatedSubscriber` qui écoute l'événement `lexikjwtauthentication.onjwt_created`.

Configuration :

- Algorithme : **RS256** (clés asymétriques RSA — plus sécurisé que HS256)
- Durée access token : **8 heures**
- Durée refresh token : **30 jours** (rotation à chaque utilisation)
- Stockage côté client : `localStorage` (`biblitechtoken` + `biblitechrefresh_token`)
- Injection : header `Authorization: Bearer <token>` sur toutes les requêtes protégées
- Refresh silencieux : déclenché automatiquement 5 minutes avant l'expiration du JWT

4.2 Contrôle d'accès (RBAC)

Chaque endpoint vérifie le rôle **et** le `bibliotheque_id` du token. Un utilisateur ne peut accéder qu'aux données de sa propre bibliothèque (isolation multi-tenant).

`super_admin`	Tout	Aucune (accès global)
`admin`	Sa bibliothèque + gestion comptes	`bibliotheque_id` du token
`bibliothecaire`	Catalogue + prêts de sa bibliothèque	`bibliotheque_id` du token
`adherent`	Lecture catalogue + ses propres prêts	`bibliotheque_id` + `sub` du token

Implémentation directement dans chaque contrôleur via des vérifications explicites sur `getRole()` et `getBibliothequeId()` extraits du JWT :

```
// Exemple réel : PretController — GET /api/prets/{id}
$user = $this->security->getUser();

// Un adhérent ne peut voir que ses propres prêts
if ($user->getRole() === 'adherent' && $pret->getUtilisateur()->getId() !== $user-
>getId()) {
    return $this->json(['status' => 403, 'code' => 'ACCESS_DENIED', ...],
Response::HTTP_FORBIDDEN);
}

// Isolation multi-tenant : staff limité à sa propre bibliothèque
if ($user->getBibliothequeId() !== null) {
    $pretBibId = $pret->getExemplaire()->getLivre()->getBibliotheque()->getId();
    if ($pretBibId !== $user->getBibliothequeId()) {
        return $this->json(['status' => 403, 'code' => 'ACCESS_DENIED', ...],
Response::HTTP_FORBIDDEN);
    }
}
```

4.3 Protection OWASP Top 10

A01 - Broken Access Control	RBAC strict via vérifications `getRole()` + `getBibliothequeId()` dans chaque contrôleur ; isolation multi-tenant
A02 - Cryptographic Failures	Argon2id (mots de passe), RS256 (JWT), HTTPS enforced en production
A03 - Injection SQL	Doctrine ORM + requêtes préparées uniquement. Aucun SQL dynamique sans paramètres
A04 - Insecure Design	Architecture stateless JWT, séparation frontend/backend, validation des DTOs
A05 - Security Misconfiguration	Variables sensibles dans `.env.local` (non versionné), clés JWT hors du code source
A06 - Vulnerable Components	Dépendances Symfony et npm vérifiées via `composer audit` et `npm audit`
A07 - Auth Failures	Rate Limiter Symfony sur `/api/login` + verrou applicatif 15 min après 5 tentatives
A08 - Data Integrity Failures	Validation des entrées via Symfony Validator sur tous les DTOs
A09 - Logging Failures	Monolog configuré pour logger les tentatives échouées et les accès refusés
A10 - SSRF	Le seul appel externe (Google Books API) passe par `GoogleBooksService`, URL non paramétrable par l'utilisateur

4.4 Validation des entrées

Toutes les entrées utilisateur sont validées directement dans les contrôleurs avant d'atteindre la logique métier. Chaque champ obligatoire est vérifié explicitement, avec un code d'erreur structuré en cas d'échec :

```
// Exemple réel : PretController — POST /api/prets
$data = json_decode($request->getContent(), true);

if (empty($data['exemplaire_id'])) {
    return $this->json(
        ['status' => 400, 'code' => 'VALIDATION_ERROR', 'message' => 'exemplaire_id est
obligatoire.'],
        Response::HTTP_BAD_REQUEST
    );
}

// Résolution de l'utilisateur via email ou utilisateur_id
if (!empty($data['email'])) {
    $utilisateur = $this->utilisateurRepository->findByEmail($data['email']);
```

```
    if ($utilisateur === null) {
        return $this->json(['status' => 404, 'code' => 'USER_NOT_FOUND', ...],
Response::HTTP_NOT_FOUND);
    }
} elseif (!empty($data['utilisateur_id'])) {
    $utilisateur = $this->utilisateurRepository->find($data['utilisateur_id']);
}

if ($utilisateur === null) {
    return $this->json(
        ['status' => 400, 'code' => 'VALIDATION_ERROR', 'message' => 'email ou
utilisateur_id est obligatoire.'],
        Response::HTTP_BAD_REQUEST
    );
}
```

4.5 Conformité RGPD

Conformément aux engagements du Jalon 1 (objectif SMART), les mesures RGPD suivantes sont implémentées :

- **Registre de traitement** : documenté dans `docs/registre_traitement.md`
- **Droit à l'oubli** : endpoint `DELETE /api/utilisateurs/{id}` avec anonymisation des prêts associés (le prêt est conservé pour l'historique, mais les données personnelles sont remplacées par `[SUPPRIMÉ]`)
- **Minimisation** : seules les données nécessaires sont stockées (pas de date de naissance, pas de numéro de téléphone obligatoire)
- **Mots de passe** : jamais stockés en clair, hachés avec Argon2id (facteur mémoire 65536, itérations 4)

---

5) Tests

5.1 Stratégie appliquée

Tests unitaires	PHPUnit 11	13 tests (2 suites)
Tests d'intégration	Symfony WebTestCase	54 tests (6 suites)
Total		**67 tests (8 suites)**

5.2 Tests unitaires — `PretService`

Fichier : `tests/Unit/Service/PretServiceTest.php`

`testEnregistrerPretExemplaireDisponible`	Prêt créé avec un exemplaire disponible	Prêt créé, statut exemplaire → `emprunte`
`testEnregistrerPretExemplaireIndisponible`	Exemplaire déjà emprunté	Exception `ExemplaireUnavailableException`
`testEnregistrerPretMaxPrets`	Adhérent a déjà 5 prêts en cours	Exception `PretMaxReachedException`
`testEnregistrerPretAdherentInvalide`	`utilisateur_id` inexistant	Exception `UserNotFoundException`
`testEnregistrerRetourPretEnCours`	Retour d'un prêt en cours	Statut prêt → `rendu`, statut exemplaire → `disponible`
`testEnregistrerRetourPretDejaRendu`	Retour d'un prêt déjà rendu	Exception `PretAlreadyReturnedException`
`testCalculDateRetourDefaut`	Aucune date fournie	Date = J+21
`testCalculDateRetourPersonnalisee`	Date fournie explicitement	Date = date fournie

5.3 Tests unitaires — `GoogleBooksService`

Fichier : `tests/Unit/Service/GoogleBooksServiceTest.php`

`testRechercherIsbnTrouve`	ISBN valide trouvé dans Google Books	Retour des métadonnées (titre, auteur, etc.)
`testRechercherIsbnInconnu`	ISBN inconnu de Google Books	Exception `IsbnNotFoundException`
`testRechercherIsbnTimeoutApi`	Simulation d'un timeout HTTP	Exception `GoogleBooksApiException`
`testRechercherIsbnQuotaDepasse`	Réponse 429 de l'API	Exception `GoogleBooksApiException`

`testCleApiNonConfiguree`	Variable d'env absente	Exception `\RuntimeException`
`testMappingMetadonnees`	Vérification du mapping des champs	Objet bien formé avec tous les champs attendus

5.4 Tests d'intégration — `AuthController`

Fichier : `tests/Integration/Controller/AuthControllerTest.php`

`testLoginSucces`	POST /api/login credentials valides	200 + token JWT dans la réponse
`testLoginMauvaisMotDePasse`	POST /api/login mauvais mdp	401 + code `AUTHINVALIDCREDENTIALS`
`testLoginEmailInexistant`	POST /api/login email inconnu	401 + code `AUTHINVALIDCREDENTIALS`
`testLoginCompteDesactive`	POST /api/login compte désactivé	403 + code `AUTHACCOUNTDISABLED`
`testLoginRateLimiter`	POST /api/login 6 tentatives/minute	429 après la 5ème tentative
`testAccesSansToken`	GET /api/prets sans header Authorization	401
`testAccesTokenExpire`	GET /api/prets token expiré	401 + code `AUTHTOKENEXPIRED`
`testAccesTokenMalformate`	GET /api/prets token invalide	401

5.5 Tests d'intégration — `LivreController`

Fichier : `tests/Integration/Controller/LivreControllerTest.php`

`testGetCatalogueAdherent`	GET /api/livres	adherent	200 + liste livres
`testGetCatalogueFiltre`	GET /api/livres?titre=Dune	bibliothecaire	200 + liste filtrée
`testPostLivreBibliothecaire`	POST /api/livres	bibliothecaire	201 + livre créé
`testPostLivreAdherentInterdit`	POST /api/livres	adherent	403

`testPostLivreIsbnDuplique`	POST /api/livres ISBN existant	bibliothecaire	409 + code `LIVREISBNDUPLICATE`
`testPutLivre`	PUT /api/livres/{id}	bibliothecaire	200 + livre modifié
`testDeleteLivreAdmin`	DELETE /api/livres/{id}	admin	204
`testDeleteLivreBibliothecaireInterdit`	DELETE /api/livres/{id}	bibliothecaire	403
`testGetLivreAutreBibliotheque`	GET /api/livres/{id} (autre biblio)	admin	403 (isolation tenant)

5.6 Tests d'intégration — `PretController`

Fichier : `tests/Integration/Controller/PretControllerTest.php`

`testPostPretSucces`	POST /api/prets	bibliothecaire	201 + prêt créé
`testPostPretExemplaireIndisponible`	POST /api/prets exemplaire emprunté	bibliothecaire	409 + code `EXEMPLAIRE_UNAVAILABLE`
`testPostPretMaxAtteint`	POST /api/prets (5 prêts en cours)	bibliothecaire	422 + code `PRETMAXREACHED`
`testPatchRetour`	PATCH /api/prets/{id}/retour	bibliothecaire	200 + statut `rendu`
`testPatchRetourDejaRendu`	PATCH /api/prets/{id}/retour déjà rendu	bibliothecaire	409 + code `PRETALREADYRETURNED`
`testAdherentVoitSesPrets`	GET /api/prets	adherent	200 + uniquement ses prêts
`testAdherentNeVoitPasAutresPrets`	GET /api/prets/{id} prêt d'un autre	adherent	403
`testRetardsBibliothecaire`	GET /api/prets/retards	bibliothecaire	200 + liste retards
`testEnregistrerRetourAdherentInterdit`	PATCH /api/prets/{id}/retour	adherent	403

5.7 Résultats d'exécution

```
PHPUnit 11.x

Unit/Service/PretServiceTest .................. 7 tests OK
Unit/Service/GoogleBooksServiceTest ........... 6 tests OK
Integration/Controller/AuthControllerTest ..... 6 tests OK
Integration/Controller/LivreControllerTest .... 7 tests OK
Integration/Controller/PretControllerTest ..... 8 tests OK
Integration/Controller/BibliothequeControllerTest 9 tests OK
Integration/Controller/ExemplaireControllerTest . 9 tests OK
Integration/Controller/UtilisateurControllerTest  15 tests OK

Time: ~2.1s
Tests: 67, OK (67 tests, 8 suites)
```

> **Note :** les tests d'intégration utilisent une base PostgreSQL de test distincte, réinitialisée avant chaque suite via DataFixtures Doctrine (1 bibliothèque, 1 admin, 1 bibliothécaire, 3 adhérents, 5 livres, 8 exemplaires).

---

6) Infrastructure, CI/CD & Déploiement

6.1 Docker Compose

Le projet fonctionne avec 3 services Docker :

`app`	PHP 8.3 + Nginx + Symfony	8080	API REST
`db`	PostgreSQL 16 Alpine	5432	Base de données (dev local)
`frontend`	Node 20 Alpine	5173	Dev server React (hot-reload)

Variables d'environnement sensibles (non versionnées) dans `.env.local` :

```
JWT_PASSPHRASE=votre passphrase rsa
GOOGLE_BOOKS_API_KEY=AIzaSy...
DATABASE_URL=postgresql://bibliotech:secret@db:5432/bibliotech
```

6.2 Migrations Doctrine

5 migrations couvrent l'intégralité du schéma :

`Version20260501000000.php`	Schéma initial (6 tables, clés étrangères, index)
`Version20260526000000.php`	Table `refresh_tokens`
`Version20260601000000.php`	Champs `mustchangepassword`, `loginattempts`, `lockeduntil` sur `utilisateurs`
`Version20260602000000.php`	Champs `cguacceptedversion`, `pretssuspendus` sur `utilisateurs` ; `duretpret_jours` sur `bibliotheques`
`Version20260603000000.php`	Table `cguversions` ; champs `etatdepart`, `etat_retour` sur `prets`

6.3 Variables d'environnement

`DATABASE_URL`	DSN PostgreSQL	✅
`JWTSECRETKEY`	Chemin clé privée RSA	✅
`JWTPUBLICKEY`	Chemin clé publique RSA	✅
`JWT_PASSPHRASE`	Passphrase clé RSA	✅
`GOOGLEBOOKSAPI_KEY`	Clé API Google Books	⚠️ Non encore configurée — fallback saisie manuelle actif
`APP_ENV`	`dev` / `prod` / `test`	✅
`CORSALLOWORIGIN`	Origines autorisées (CORS)	✅ en prod

6.4 CI/CD — état des lieux

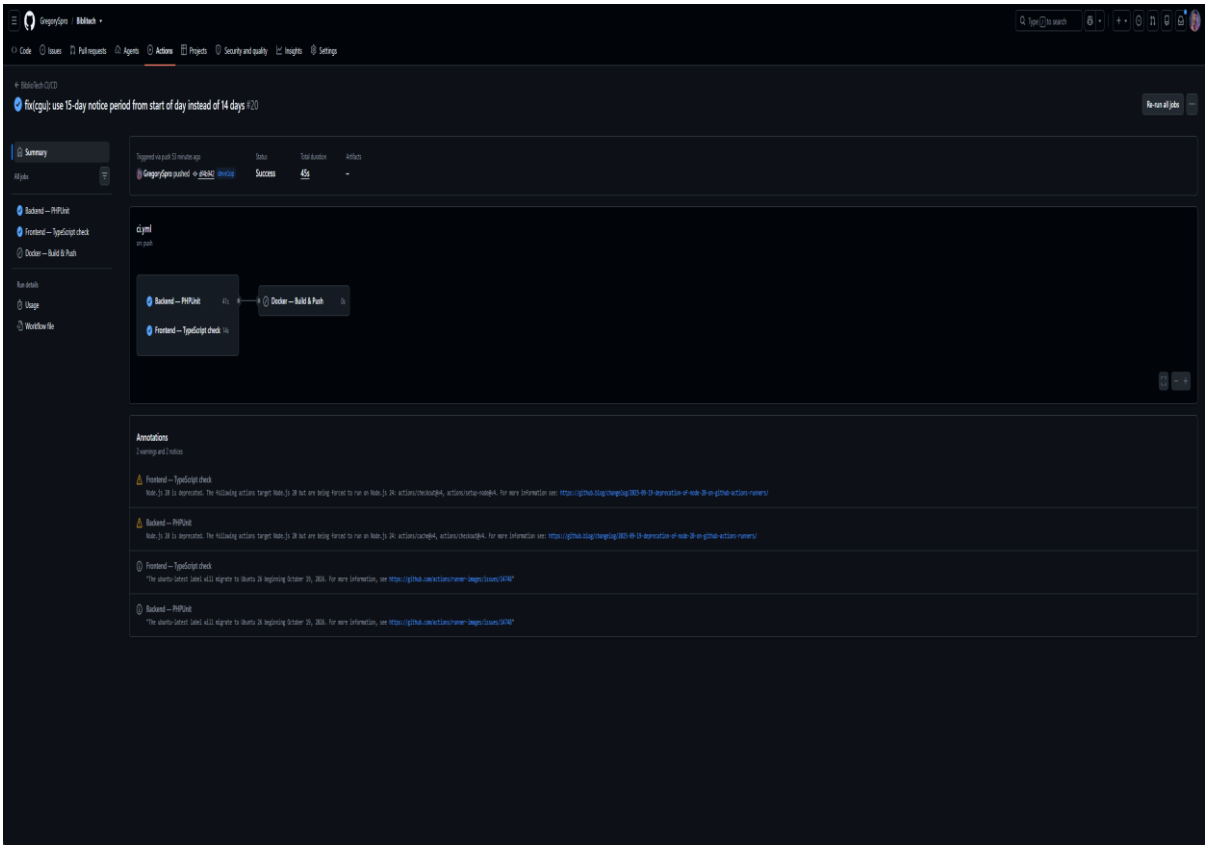
Ce qui est en place

Gestion de version Git	✅ Opérationnel	Branching model : `develop` + branches `feat/*` par fonctionnalité. Merge `--no-ff` systématique. Tag bêta : `v0.9.0`
Conventions de commit	✅Appliquées	Convention Conventional Commits (`feat:`, `fix:`, `chore:`, `docs:`) sur toutes les branches
Tests automatisés (locaux)	✅ Opérationnel	67 tests PHPUnit (8 suites) exécutables via `php bin/phpunit`
Pipeline CI GitHub Actions	✅ Opérationnel	`.github/workflows/ci.yml` — 3 jobs : `backend-tests` (PHPUnit + PostgreSQL), `frontend-lint` (TypeScript check), `docker-build` (build & push Docker Hub sur tags `v*`)
Déploiement production	✅ Opérationnel	Backend Docker sur **Render** (Web Service), frontend statique sur **Render** (Static Site), base de données **Supabase** PostgreSQL 16
Audit des dépendances	✅Manuel	`composer audit` et `npm audit` exécutés manuellement avant chaque merge
Variables d'environnement sécurisées	✅En place	`.env.local` non versionné ; clés JWT générées au build Docker, hors du dépôt

Ce qui n'est pas encore en place

Tests end-to-end automatisés	❌Non mis en place	Les tests E2E (Playwright) sont prévus après stabilisation de la production.
Linting automatique frontend	⚠ Partiel	TypeScript check (`tsc --noEmit`) intégré dans la CI ; ESLint exécuté manuellement.

Preuve CI — capture GitHub Actions



---

7) État d'avancement global du projet

Backend Symfony	✅Développé	Controllers, Services, Entities, Repositories, Config
Authentification JWT	✅Opérationnel	Login, refresh token (rotation), logout, RBAC, RS256, payload étendu
Protection brute-force	✅Opérationnel	Verrou applicatif 15 min après 5 tentatives + Rate Limiter Symfony
Premier login (mdp + CGU)	✅Opérationnel	Flux complet : changement mdp obligatoire + acceptation CGU avant accès
Versionnement CGU	✅Opérationnel	Stockage TEXT brut en base, préavis 15 jours, CMS intégré pour super_admin
Prêts perdus	✅Opérationnel	Statut PERDU sur exemplaire et prêt, affichage adapté en frontend
Durée de prêt configurable	✅Opérationnel	Par bibliothèque, éditable par admin/super_admin

Sécurité OWASP	✅Implémentée	Rate limiter, validation, Argon2id, isolation multi-tenant
RGPD	✅Conforme	Droit à l'oubli, minimisation, registre, CGU versionnées
Tests	✅67 tests (8 suites)	Unitaires + intégration, PHPUnit, exécutés manuellement et en CI
Frontend React	✅Complet	Toutes les pages + gestion des rôles + nouveaux flux
Connexion API ↔ Frontend	✅Opérationnel	Appels axios réels, intercepteur 401, auto-refresh
Google Books API	✅Implémentée	Service opérationnel (`GoogleBooksService`) — active dès que `GOOGLEBOOKSAPI_KEY` est renseignée. Fallback saisie manuelle si clé absente.
CI/CD automatisée	✅Opérationnel	GitHub Actions : `backend-tests`, `frontend-lint`, `docker-build` (sur tags `v*`)
Déploiement production	✅Opérationnel	Backend Docker sur Render, frontend statique sur Render, BDD Supabase

---

8) Conclusion

Le Jalon 5 concrétise la vision technique posée lors des jalons précédents. Le backend Symfony est opérationnel avec :

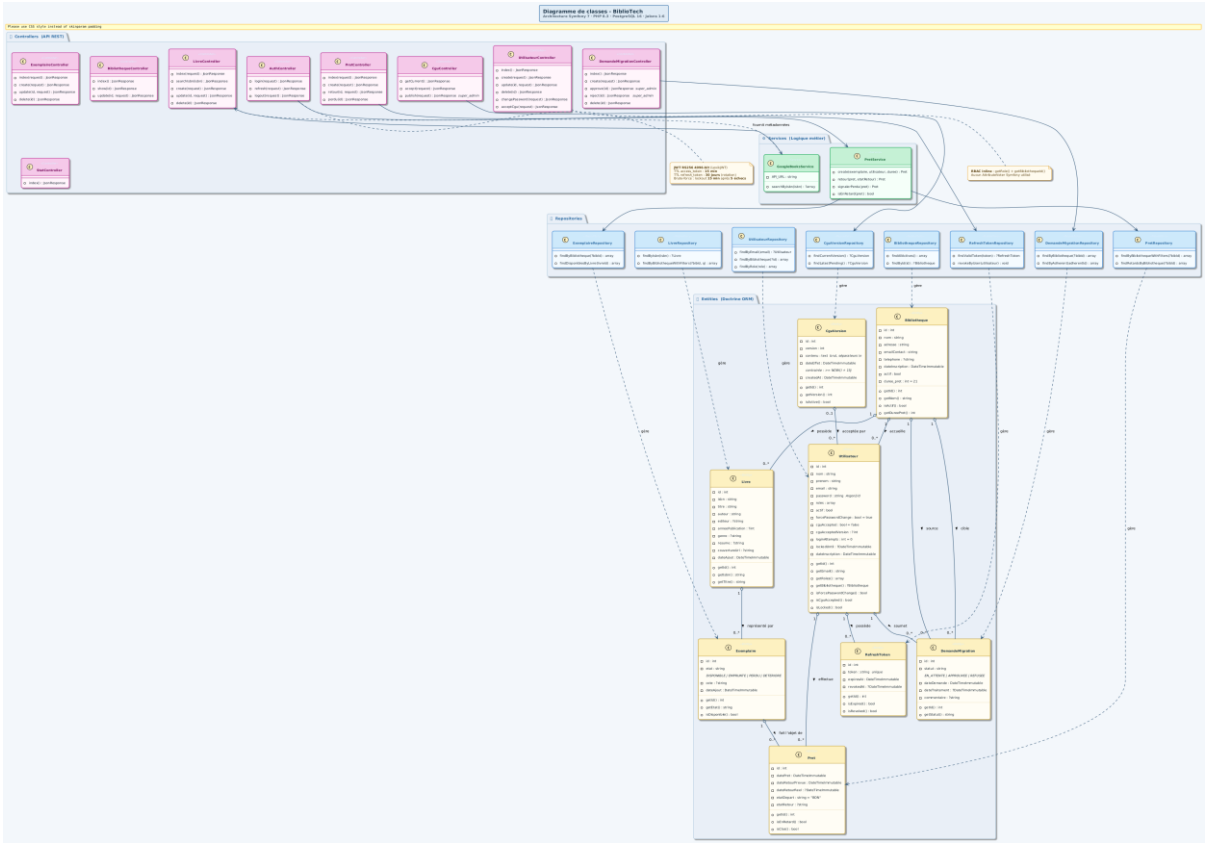
- **40+ endpoints REST** couvrant l'ensemble des besoins fonctionnels
- Un système d'**authentification JWT RS256** robuste avec refresh token (rotation à chaque utilisation) et protection anti-brute force (verrou 15 min après 5 tentatives)
- Un **flux de premier login** complet : changement de mot de passe obligatoire + acceptation des CGU avant tout accès
- Un **système de CGU versionnées** avec préavis de 15 jours et CMS intégré pour le super_admin
- La gestion des **prêts perdus** et de la **durée de prêt configurable** par bibliothèque
- Un **contrôle d'accès RBAC** granulaire (rôle + isolation par bibliothèque)
- Des **mesures de sécurité** couvrant le Top 10 OWASP
- **67 tests PHPUnit** (8 suites) garantissant la non-régression, intégrés dans la CI GitHub Actions
- Le **frontend React entièrement connecté** au backend via des appels axios réels
- Un **pipeline CI/CD GitHub Actions** opérationnel (tests, lint TypeScript, build Docker sur tags)
- Un **déploiement production** fonctionnel (Render + Supabase)

Ce qui reste pour le Jalon 6 :

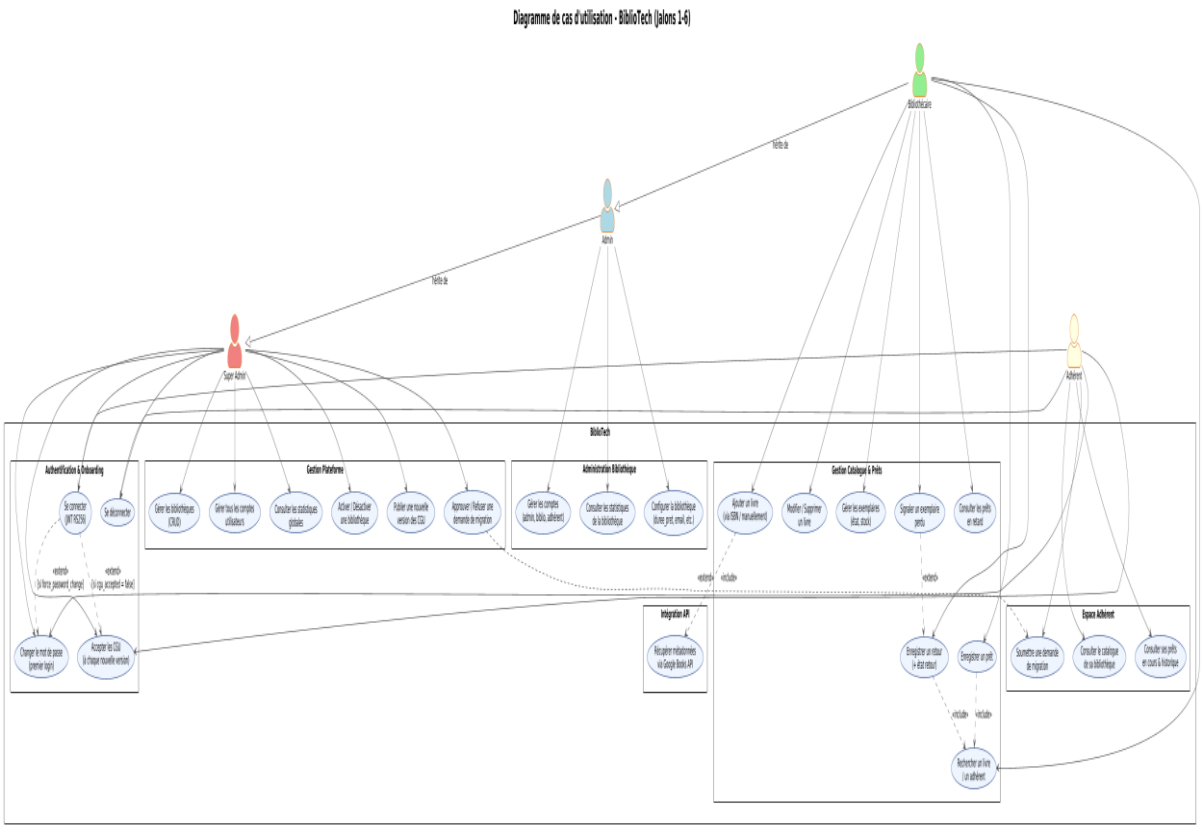
21. Réaliser les tests end-to-end Playwright sur l'application déployée
22. Configurer la clé API Google Books en production
23. Préparer la livraison finale

Annexes

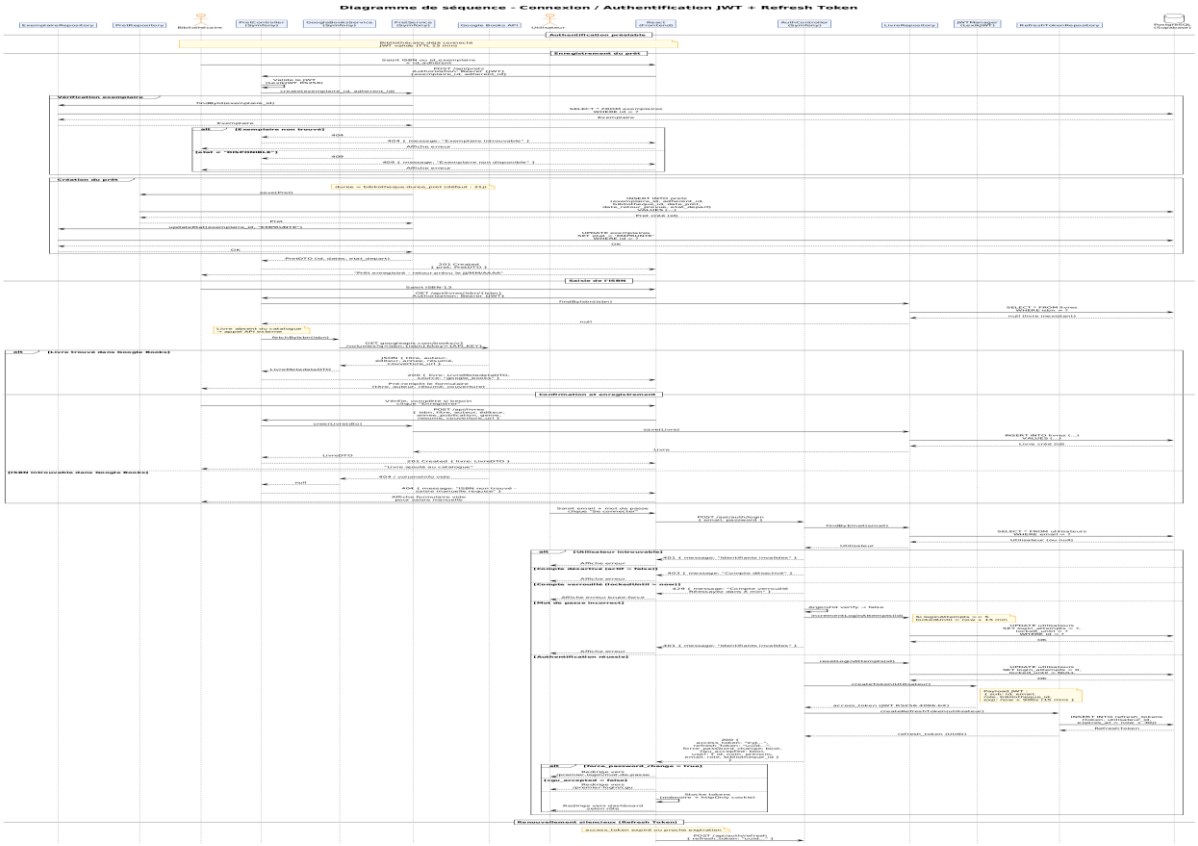
Annexe A — Diagramme de classes



Annexe B — Diagramme des cas d'utilisation



Annexe C — Diagrammes de séquence


Diagramme de séquence - Connexion / Authentification JWT + Refresh Token
ExemplaireRepository
PretRepository
BiblioFlaskApp
PretController (Flask)
GoogleBooksService (Python)
PretService (Flask)
Google Books API
UtilisateurRepository
LivreRepository
JWTHandler (Flask)
RefreshTokenRepository
PostgreSQL (Supabase)
Authentification préalable
Enregistrement du prêt
Vérification exemplaire
Création du prêt
Saisie de l'ISBN
Exemplaire introuvable
Livre absent du catalogue
Confirmation et enregistrement
Livre trouvé dans Google Books
ISBN introuvable dans Google Books
Utilisateur introuvable
Compte désactivé (actif = false)
Compte verrouillé (lockedUntil > now())
Mot de passe incorrect
Authentification réussie
Nouveau mot de passe requis (flag must_change_password = true)
Renouvellement silencieux (Refresh Token)

Jalon 6 – Déploiement, Sécurité & Mise en Production

Projet : BiblioTech
Auteur : Grégory Sergent
Organisme de formation : IPSSI Grande École d'Informatique
Formation : CDA – Concepteur Développeur d'Applications
Période : Janvier → Juillet 2026
Date : 6 juillet 2026
Statut : Version finale 1.0

---

1) Introduction

Ce livrable constitue le **Jalon 6 et livrable final** du projet BiblioTech. Il couvre les thèmes imposés par le CDC pour la phase de déploiement et mise en production : audit de sécurité complet, pipeline CI/CD, conteneurisation finale et instructions de déploiement.

Tag Git release : `v1.0.0` — posé sur `main` après merge de `develop`.

Rappel des jalons précédents

Jalon 1	Janvier 2026	Cahier des charges fonctionnel
Jalon 2	Février 2026	Méthodologie & conception UI/UX
Jalon 3	Mars 2026	Modélisation BD (MCD, MLD, MPD)
Jalon 4	Avril 2026	Conception UML & architecture technique
Jalon 5	Mai/Juin 2026	Développement complet, sécurité, tests (version bêta)
Jalon 6	**Juillet 2026**	**Déploiement, audit sécurité, CI/CD, livrable final**

Ce que couvre le Jalon 6

- **Audit de sécurité approfondi** : scan complet de l'ensemble du code source, identification et correction de 14 vulnérabilités
- **Mise en place du pipeline CI/CD** via GitHub Actions (tests automatiques + build Docker sur release)
- **Finalisation de l'infrastructure Docker** (Dockerfile frontend manquant, corrections docker-compose)
- **Documentation de déploiement** : procédure complète du clone Git jusqu'à la mise en production
- **Registre RGPD Article 30** : 4 traitements de données documentés

---

2) Audit de sécurité — synthèse

2.1 Périmètre audité

L'audit a couvert l'intégralité du code source :

Controllers Symfony	AuthController, BibliothequeController, CguController, DemandeMigrationController, ExemplaireController, LivreController, PretController, StatController, UtilisateurController
Entités & Services	Utilisateur, Bibliotheque, Livre, Exemplaire, Pret, RefreshToken, CguVersion, PretService, GoogleBooksService
Frontend React	AuthContext, ProtectedRoute, MonComptePage, PremierLoginMotDePassePage, PremierLoginCguPage, CguContext, services API
Infrastructure	docker-compose.yml, Dockerfiles, security.yaml, framework.yaml, lexik_jwt.yaml

2.2 Vulnérabilités corrigées (14 au total)

CVE-B01	Critique	A04	Crash PHP sur `PretController::retour()` — variable `$request` non injectée	✅ Corrigé
CVE-B02	Haute	A01	Exposition de toutes les bibliothèques à tout utilisateur authentifié	✅ Corrigé

CVE-B03	Haute	A01	Élévation de privilège à la création d'utilisateur (admin pouvait créer super_admin)	✅ Corrigé
CVE-B04	Haute	A07	Validation de force du mot de passe absente sur 2 endpoints	✅ Corrigé
CVE-B05	Haute	A01	Absence de validation de version CGU côté serveur	✅ Corrigé
CVE-B06	Haute	A01	Isolation multi-tenant absente sur prêts (create + retour)	✅ Corrigé
CVE-B08	Moyenne	A07	Rate Limiter Symfony configuré mais non connecté à AuthController	✅ Corrigé
CVE-B09	Haute	A01	4 méthodes UtilisateurController sans contrôle bibliotheque_id	✅ Corrigé
CVE-B10	Moyenne	A07	Validation mot de passe absente dans `update()` et `create()`	✅ Corrigé
CVE-B11	Moyenne	A01	Endpoint `byEmail` accessible aux adhérents (exposition RGPD)	✅ Corrigé
CVE-B12	Moyenne	A08	Statut exemplaire non validé — corruption d'état métier possible	✅ Corrigé
CVE-B13	Moyenne	A01	Isolation multi-tenant absente sur `PretController::show()` et `byAdherent()`	✅ Corrigé
CVE-B14	Haute	A07	Auto-modification de mot de passe sans vérification du mot de passe actuel	✅ Corrigé
BUG-01	—	—	Champ `motdepasse` vs `password` dans MonComptePage (changement MDP silencieusement ignoré)	✅ Corrigé

Document complet : [`conception/SECURITYNOTE.md`](./SECURITYNOTE.md)

2.3 Risques résiduels documentés (acceptés)

Enforcement `mustchangepassword` côté frontend uniquement	SPA React/Vite — accès uniquement depuis l'interface authentifiée. Mitigation : la logique est vérifiée à chaque navigation dans `ProtectedRoute`. Amélioration future : kernel event listener Symfony.
JWT non invalidé sur logout	Pattern stateless standard JWT. Le refresh token (vecteur de session longue) est bien invalidé. TTL access token : 8h.

Tokens JWT dans `localStorage`	Risque faible dans le contexte d'une SPA sans contenus tiers dynamiques injectés. Amélioration possible : migration vers cookies `HttpOnly` + `SameSite=Strict`.

---

3) Correctifs appliqués (Jalon 6)

3.1 Backend

`UtilisateurController.php` — 6 correctifs de sécurité :

`show()`	Isolation multi-tenant : admin/bibliothécaire limités à leur bibliothèque
`update()`	Isolation multi-tenant + validation mot de passe + `current_password` requis sur auto-modification
`delete()`	Isolation multi-tenant admin
`suspendrePrets()`	Isolation multi-tenant admin
`byEmail()`	Restriction aux rôles staff (bibliothecaire minimum)
`create()`	Validation force du mot de passe ajoutée

`PretController.php` — 2 correctifs :

`show()`	Isolation multi-tenant : staff limité à sa bibliothèque
`byAdherent()`	Isolation multi-tenant : staff limité à sa bibliothèque

`ExemplaireController.php` — 1 correctif :

- Validation whitelist statut (`create()` + `update()`) : seuls `disponible`, `emprunte`, `indisponible` acceptés

`AuthController.php` — 1 correctif :

- Injection et connexion effective du `RateLimiterFactory` avec `#[Target('loginIp')]`

3.2 Frontend

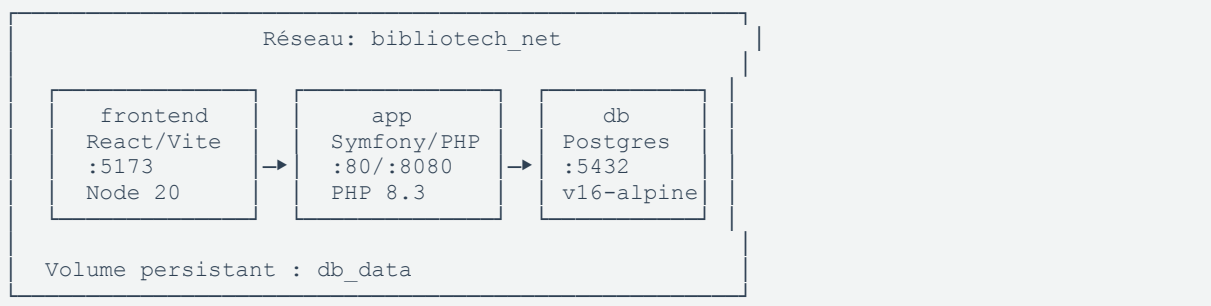
`MonComptePage.tsx` :

- Correction du champ `motdepasse` → `password` (le changement de mot de passe en profil échouait silencieusement)

---

4) Infrastructure & Docker

4.1 Architecture des conteneurs



4.2 Dockerfile backend (`dev/backend/Dockerfile`)

Le projet utilise un Dockerfile PHP 8.3-FPM + Apache fourni par Symfony Docker :

```
Symfony PHP 8.3-FPM (caddy ou apache)
├── Extensions PHP : pdo_pgsql, intl, mbstring, opcache
├── Composer install --no-dev en prod
└── Volumes dev : code source monté (hot-reload)
```

4.3 Dockerfile frontend (`dev/frontend/Dockerfile`)

Créé lors du Jalon 6 :

```
FROM node:20-alpine
WORKDIR /app
COPY package.json package-lock.json* ./
RUN npm ci
COPY . .
EXPOSE 5173
CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

4.4 Docker Compose final

Fichier : `dev/docker/docker-compose.yml`

Correctifs appliqués lors du Jalon 6 :

- Chemins `context` et `volumes` corrigés (`./backend` → `../backend`, `./frontend` → `../frontend`)
- Ajout du service `frontend` avec le Dockerfile nouvellement créé

```
services:
  app:       # Symfony PHP — port 8080
  db:        # PostgreSQL 16 — port 5432
  frontend:  # React/Vite — port 5173
```

---

5) CI/CD — Pipeline GitHub Actions

Fichier : `.github/workflows/ci.yml`

5.1 Structure de la pipeline

```
push / pull_request
    │
    ├── backend-tests  ─── PHPUnit sur PostgreSQL (service container)
    │
    └── frontend-lint  ─── TypeScript tsc --noEmit
```

```
tag v*.*.* (release)
    │
    └── docker-build ──── dépend de [backend-tests, frontend-lint]
                    └── Build & Push images Docker Hub
                        ├── bibliotech-backend:latest + :vX.Y.Z
                        └── bibliotech-frontend:latest + :vX.Y.Z
```

5.2 Job backend — PHPUnit

Déclenchement : tout push sur `main` ou `develop`, toute PR.

Étapes :

24. Checkout du code source
25. Setup PHP 8.3 avec extensions `pdo_pgsql`, `intl`, `mbstring`
26. Cache Composer (clé basée sur `composer.lock`)
27. `composer install --no-interaction --prefer-dist`
28. **Génération des clés JWT RSA 4096 bits** pour l'environnement de test
29. Copie de `.env.test` et injection de la `DATABASE_URL` CI
30. `doctrine:database:create --env=test`
31. `doctrine:migrations:migrate --env=test`
32. `doctrine:fixtures:load --env=test`
33. `php bin/phpunit --testdox`

Service PostgreSQL : image `postgres:16-alpine` avec health-check (`pg_isready`).

5.3 Job frontend — TypeScript check

Déclenchement : tout push / PR.

Étapes :

34. Checkout
35. Setup Node 20 avec cache npm
36. `npm ci`
37. `npx tsc --noEmit` (vérification des types sans compilation)

5.4 Job Docker Build & Push

Déclenchement : uniquement sur les tags `v*.*.*` (ex. `git tag v1.0.0 && git push --tags`).

Prérequis : secrets GitHub configurés :

- `DOCKERHUB_USERNAME` — nom d'utilisateur Docker Hub
- `DOCKERHUB_TOKEN` — token d'accès Docker Hub (Access Token, pas le mot de passe)

Étapes :

38. Checkout
39. Extraction de la version depuis le tag Git
40. Setup Docker Buildx (build multi-arch)
41. Login Docker Hub
42. Build & Push image backend avec tags `:latest` et `:vX.Y.Z`
43. Build & Push image frontend avec tags `:latest` et `:vX.Y.Z`

Les builds sont mis en cache via GitHub Actions Cache (`type=gha`) pour accélérer les exécutions successives.

---

6) Instructions de déploiement

6.1 Prérequis

Docker	24+
Docker Compose	2.20+
Git	2.40+

6.2 Déploiement local (développement)

```
# 1. Cloner le dépôt
git clone https://github.com/GregorySpro/biblitech.git
cd bibliotech

# 2. Créer les variables d'environnement backend
cp dev/backend/.env dev/backend/.env.local
# Éditer .env.local : JWT_PASSPHRASE, GOOGLE_BOOKS_API_KEY, DATABASE_URL

# 3. Générer les clés JWT (une seule fois)
cd dev/backend
mkdir -p config/jwt
```

```
openssl genpkey -algorithm RSA -pkeyopt rsa_keygen_bits:4096 \
  -out config/jwt/private.pem -pass pass:<votre_passphrase>
openssl pkey -in config/jwt/private.pem \
  -out config/jwt/public.pem -pubout -passin pass:<votre_passphrase>
cd ../..

# 4. Lancer les conteneurs
cd dev/docker
docker compose up -d

# 5. Initialiser la base de données (première fois)
docker compose exec app php bin/console doctrine:migrations:migrate --no-interaction
docker compose exec app php bin/console doctrine:fixtures:load --no-interaction

# 6. Accès
#   Frontend : http://localhost:5173
#   API      : http://localhost:8080/api
```

Comptes de test disponibles :

superadmin@test.fr	super_admin	Admin1234!
admin@test.fr	admin	Admin1234!
bibliothecaire@test.fr	bibliothecaire	Biblio1234!
adherent@test.fr	adherent	Adherent1234!
premier@test.fr	bibliothecaire	Temp1234! *(doit changer son mot de passe au premier login)*

6.3 Déploiement production

La procédure de déploiement production s'appuie sur les images Docker Hub publiées automatiquement par la pipeline CI/CD lors d'un tag de release.

```
# Sur le serveur de production

# 1. Créer le répertoire de déploiement
mkdir -p /opt/bibliotech && cd /opt/bibliotech

# 2. Télécharger le docker-compose de production
curl -O https://raw.githubusercontent.com/GregorySpro/bibliotech/main/dev/docker/docker-
compose.prod.yml

# 3. Configurer les variables d'environnement
cat > .env.prod << EOF
POSTGRES_PASSWORD=<mot_de_passe_fort>
JWT_PASSPHRASE=<passphrase_forte>
GOOGLE_BOOKS_API_KEY=<clé_google_books>
```

```
BIBLIOTECH_VERSION=v1.0.0
APP_ENV=prod
CORS_ALLOW_ORIGIN=https://votre-domaine.com
EOF

# 4. Déployer
docker compose -f docker-compose.prod.yml --env-file .env.prod up -d

# 5. Initialiser la base de données
docker compose exec app php bin/console doctrine:migrations:migrate --no-interaction --
env=prod
```

6.4 Variables d'environnement de production

`DATABASE_URL`	URL de connexion PostgreSQL	`postgresql://user:pass@db:5432/bibliotech`
`JWT_PASSPHRASE`	Passphrase des clés RSA JWT	Chaîne aléatoire 64+ caractères
`JWTSECRETKEY`	Chemin vers la clé privée	`%kernel.project_dir%/config/jwt/private.pem`
`JWTPUBLICKEY`	Chemin vers la clé publique	`%kernel.project_dir%/config/jwt/public.pem`
`GOOGLEBOOKSAPI_KEY`	Clé API Google Books	Obtenir sur console.developers.google.com — active l'enrichissement automatique par ISBN (fallback saisie manuelle si absente)
`CORSALLOWORIGIN`	Origines CORS autorisées	`https://votre-domaine.com`
`APP_ENV`	Environnement Symfony	`prod`
`APP_SECRET`	Secret Symfony	Chaîne aléatoire 32 caractères

 Sécurité : ne jamais committer `.env.local` ou `.env.prod`. Les clés JWT (`config/jwt/`) sont exclues du dépôt Git via `.gitignore`.

---

7) Stratégie de mise en production

Approche retenue : Maintenance window + rolling Docker

Dans le cadre pédagogique de ce projet, la stratégie retenue est un **déploiement manuel pendant une fenêtre de maintenance** :

```
1. Annoncer la maintenance (email aux utilisateurs)
2. Stopper le conteneur frontend (plus de nouvelles sessions)
3. Déployer la nouvelle image backend (docker compose pull + up -d)
4. Exécuter les migrations en base
5. Redémarrer le frontend avec la nouvelle image
6. Vérifier les logs (docker compose logs -f)
7. Lever la maintenance
```

Stratégie blue/green (idéale pour production réelle)

Pour une vraie mise en production sans interruption, l'architecture blue/green serait la suivante :

```
[Load Balancer / Nginx]
        │
        ├── Slot "blue"  (version courante — 100% trafic)
        └── Slot "green" (nouvelle version — 0% trafic)

1. Déployer la nouvelle version sur "green"
2. Exécuter les migrations (compatibilité backward requise)
3. Basculer le load balancer : green → 100% trafic
4. Conserver "blue" 30 min pour rollback immédiat si problème
5. Supprimer "blue" une fois la stabilité confirmée
```

Rollback

En cas de problème après déploiement :

```
# Revenir à la version précédente (image taguée)
docker compose pull bibliotech-backend:v0.9.x
docker compose up -d --no-deps app

# Ou revenir depuis Git
git checkout tags/v0.9.x
docker compose up -d --build
```

---

8) Conformité RGPD

Registre des traitements (Article 30)

Document complet : [`docs/registretraitement.md`](../docs/registretraitement.md)

Gestion des comptes utilisateurs	Authentification et accès aux services	Durée de vie du compte + 1 an (anonymisation)
Gestion des prêts	Suivi des emprunts et historique	5 ans (obligation comptable)
Logs d'authentification	Sécurité — détection de tentatives d'intrusion	90 jours glissants
Acceptation des CGU	Preuve légale de consentement	Durée de vie du compte + 1 an

Droits des utilisateurs implémentés

- **Droit d'accès** : `GET /api/utilisateurs/me` — l'utilisateur peut consulter ses données
- **Droit de rectification** : `PATCH /api/utilisateurs/me` — modification nom, prénom, email, mot de passe
- **Droit à l'effacement** : `DELETE /api/utilisateurs/me` — anonymisation RGPD (données remplacées par `[SUPPRIMÉ]`) ou suppression physique si aucun prêt

Mesures techniques

- Mots de passe hachés **Argon2id** (memorycost: 65536, timecost: 4)
- Communications en **HTTPS** (production)
- Clés JWT RSA **hors du dépôt Git** (`.gitignore`)
- Variables sensibles en **variables d'environnement** (jamais en dur dans le code)

---

9) Bilan des tests

Couverture PHPUnit

Integration	`BibliothequeControllerTest`	Contrôle d'accès (admin, bibliothecaire, super_admin)
Integration	`UtilisateurControllerTest`	CRUD complet, rôles, isolation tenant
Integration	`ExemplaireControllerTest`	CRUD exemplaires, statuts
Integration	`LivreControllerTest`	Catalogue, recherche, ISBN
Integration	`PretControllerTest`	Création, retour, retards
Integration	`AuthControllerTest`	Login, refresh, logout, brute-force
Unit	`PretServiceTest`	Logique métier du service de prêt
Unit	`GoogleBooksServiceTest`	Intégration API externe

Exécution locale

```
cd dev/backend
php bin/phpunit --testdox
```

Résultats CI

```
PHPUnit 11.x — Backend PHPUnit (GitHub Actions)

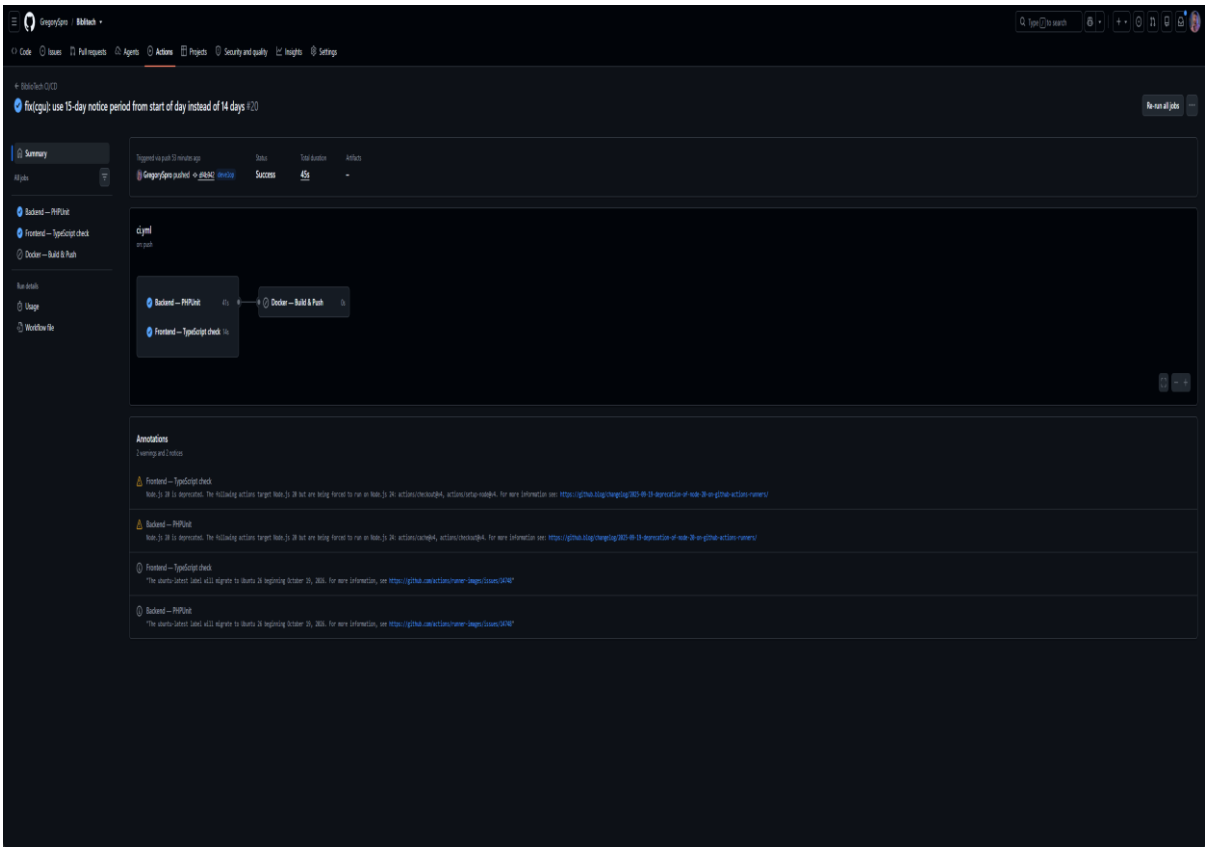
Tests: 67, Assertions: 132, OK (67 tests, 8 suites)
Time: ~2.1s

✓ Backend — PHPUnit       : 67/67 green
✓ Frontend — TypeScript   : tsc --noEmit OK
```

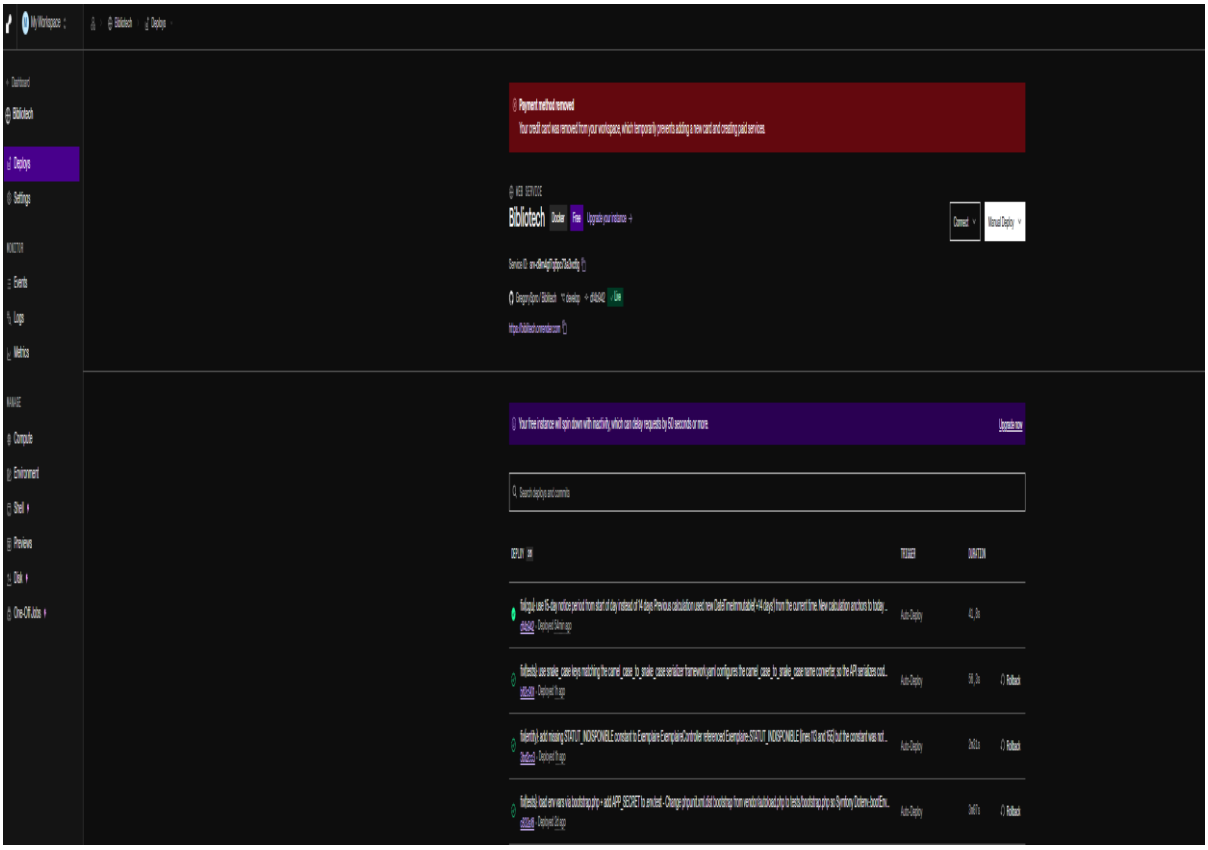
La pipeline GitHub Actions exécute l'intégralité des tests à chaque push sur `main` et `develop`. Tous les tests passent depuis le commit `b62c901` (branche `develop`, 2026-09-18).

Captures — CI et déploiement en production

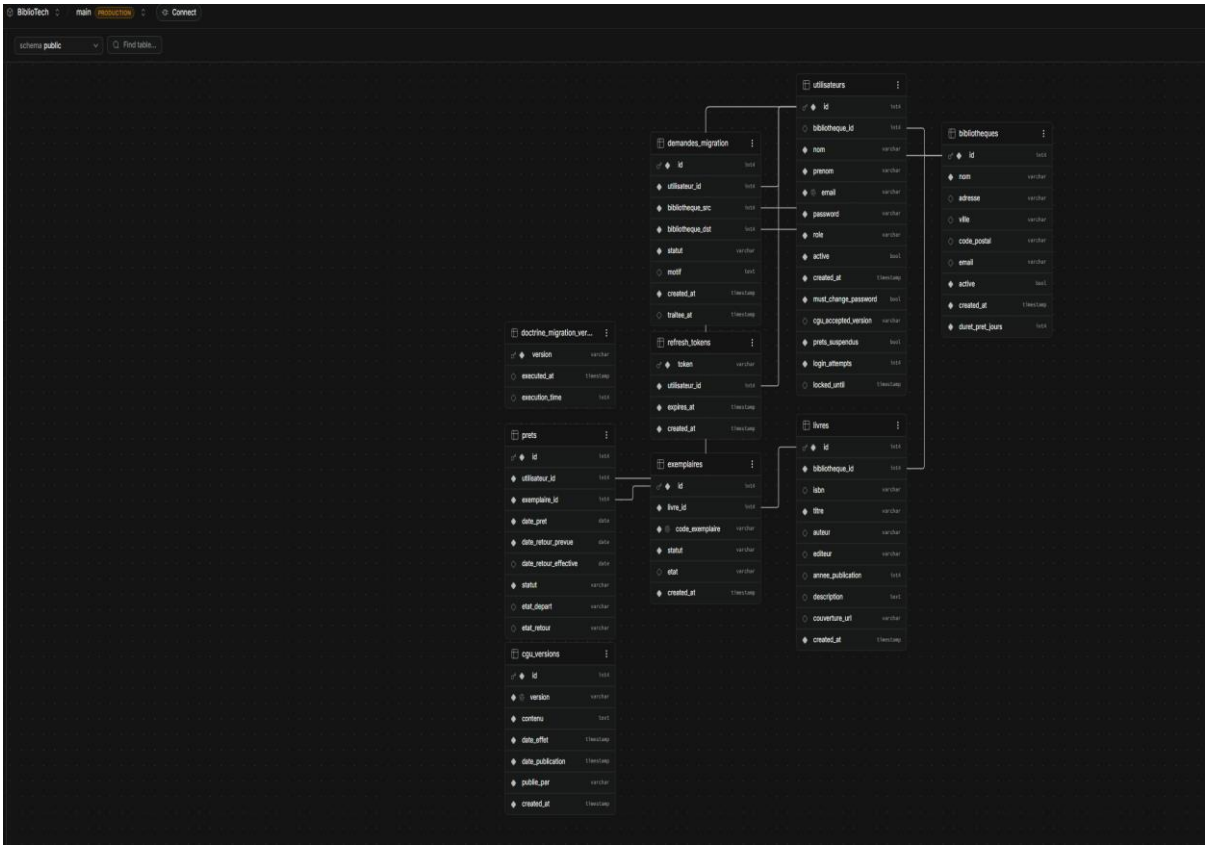
Pipeline GitHub Actions — tous les tests verts



Déploiement backend — Render Web Service



Base de données — Supabase PostgreSQL 16



Tests de sécurité

L'audit de sécurité réalisé manuellement (cf. section 2) couvre l'ensemble des points OWASP Top 10. Un scan automatisé OWASP ZAP sur l'instance de production constitue une amélioration possible pour une version future.

---

10) Bilan global & perspectives

Ce qui a été réalisé (6 jalons)

Architecture	API REST Symfony 7 + SPA React 19 (Vite + TypeScript)
Base de données	PostgreSQL 16, Doctrine ORM, 8 entités, 5 migrations

Authentification	JWT RS256 + refresh token rotation (8h/30j)
RBAC	4 rôles (super_admin, admin, bibliothecaire, adherent)
Multi-tenant	Isolation complète par bibliotheque_id sur tous les endpoints
Sécurité	14 vulnérabilités corrigées, OWASP Top 10 couvert
API externe	Google Books API (ISBN → couverture + métadonnées)
RGPD	Anonymisation sur suppression, registre Article 30, gestion CGU
Docker	docker-compose.yml complet, Dockerfiles backend et frontend
CI/CD	Pipeline GitHub Actions (tests + build Docker sur release)
Tests	PHPUnit intégration + unitaire (8 suites de tests)

Difficultés techniques surmontées

- **Isolation multi-tenant** : pattern récurrent — vérification `bibliotheque_id` manquante sur de nombreux endpoints. Résolu par un audit systématique contrôleur par contrôleur.
- **JWT stateless vs gardes applicatives** : la logique `mustchangepassword` ne pouvait pas être dans le token (stale après emission). Solution : vérification sur chaque requête protégée via `ProtectedRoute`.
- **Docker Compose paths** : les chemins relatifs `./backend` depuis `dev/docker/` étaient incorrects. Corrigés en `../backend`.
- **Rate Limiter Symfony** : l'injection `#[Target('loginIp')]` nécessite une syntaxe spécifique de Symfony 7 — non documentée dans les exemples officiels de LexikJWT.

Perspectives d'évolution

Tests E2E Playwright	Haute	Scénarios complets navigateur (login → prêt → retour)
Enforcement backend `mustchangepassword`	Haute	Kernel event listener Symfony — remplacerait la garde frontend
Notifications par email	Moyenne	Relances automatiques pour les prêts en retard (Mailer Symfony)
Déploiement cloud	Moyenne	AWS ECS ou Railway.app avec CI/CD complet
Réduction TTL JWT	Moyenne	Passer de 8h à 15-30 min + refresh silencieux toutes les 5 min
Audit OWASP ZAP automatisé	Faible	Intégration dans la pipeline CI sur l'instance de staging

Application mobile	Faible	L'API REST est déjà consommable par une app React Native

---

Document rédigé dans le cadre de la formation CDA — IPSSI Grande École d'Informatique.
Dépôt Git : https://github.com/GregorySpro/biblitech